hierarchial_model

```r
suppressPackageStartupMessages(library(tidyverse))
suppressPackageStartupMessages(library(ggplot2))
suppressPackageStartupMessages(library(dplyr))
suppressPackageStartupMessages(library(bayesplot))
suppressPackageStartupMessages(library(cmdstanr))
suppressPackageStartupMessages(library(f1dataR))
```

```r
## The dataset full_data.csv is included in the repository.
##If you need to regenerate it from scratch using the f1dataR package,
##use the following code (note this takes a long time to run):

# full_data <- map_dfr(2019:2024, function(year) {
#   schedule <- load_schedule(season = year)
#   map_dfr(seq_len(nrow(schedule)), function(round) {
#     tryCatch(
#       load_results(season = year, round = round) %>%
#         mutate(
#           season = year,
#           round = round,
#           race_name = schedule$race_name[round],
#           circuit = schedule$circuit_id[round],
#           top_speed_kph = as.numeric(top_speed_kph),
#           fastest_rank = as.integer(fastest_rank)
#         ),
#       error = function(e) {
#         message("Skipping season ", year, " round ", round, ": ", e$message)
#         NULL
#       }
#     )
#   })
# })
#
# write_csv(full_data, "full_data.csv")
```

```r
## Loading and preprocessing data

data <- read.csv("full_data.csv")

convert_to_seconds <- function(x) {
  if (is.numeric(x)) return(x)
  x <- as.character(x)
  sapply(x, function(val) {
    if (grepl("^[+-]?\\d+\\.?\\d*$", val)) {
      return(as.numeric(val))
    }
    if (!grepl("^[+-]", val) && grepl(":", val)) {
      return(0)
```

```r
    }
    sign <- ifelse(startsWith(val, "-"), -1, 1)
    val <- gsub("^[+-]", "", val)
    parts <- strsplit(val, ":")[[1]]
    if (length(parts) == 2) {
      return(sign * (as.numeric(parts[1]) * 60 + as.numeric(parts[2])))
    } else if (length(parts) == 3) {
      return(sign * (as.numeric(parts[1]) * 3600 + as.numeric(parts[2]) * 60 + as.numeric(parts[3])))
    } else {
      return(NA)
    }
  }, USE.NAMES = FALSE)
}

data <- data %>%
  mutate(gap = convert_to_seconds(gap),
         grid = as.integer(grid)) %>%
  filter(
    status == "Finished",
    !is.na(grid),
    grid > 0,
    !is.na(gap))

# head(data)
```

```r
## Indexing circuit for stan

circuit_levels <- sort(unique(data$circuit))

data <- data |>
  mutate(circuit_id = as.integer(factor(circuit, levels = circuit_levels)))

N_circuits <- length(circuit_levels)
N_obs      <- nrow(data)

cat("Total observations:", N_obs, "\n")
```

```
## Total observations: 1464
```

```r
cat("Number of circuits:", N_circuits, "\n")
```

```
## Number of circuits: 31
```

```r
cat("Circuit index range:", min(data$circuit_id), "to", max(data$circuit_id), "\n")
```

```
## Circuit index range: 1 to 31
```

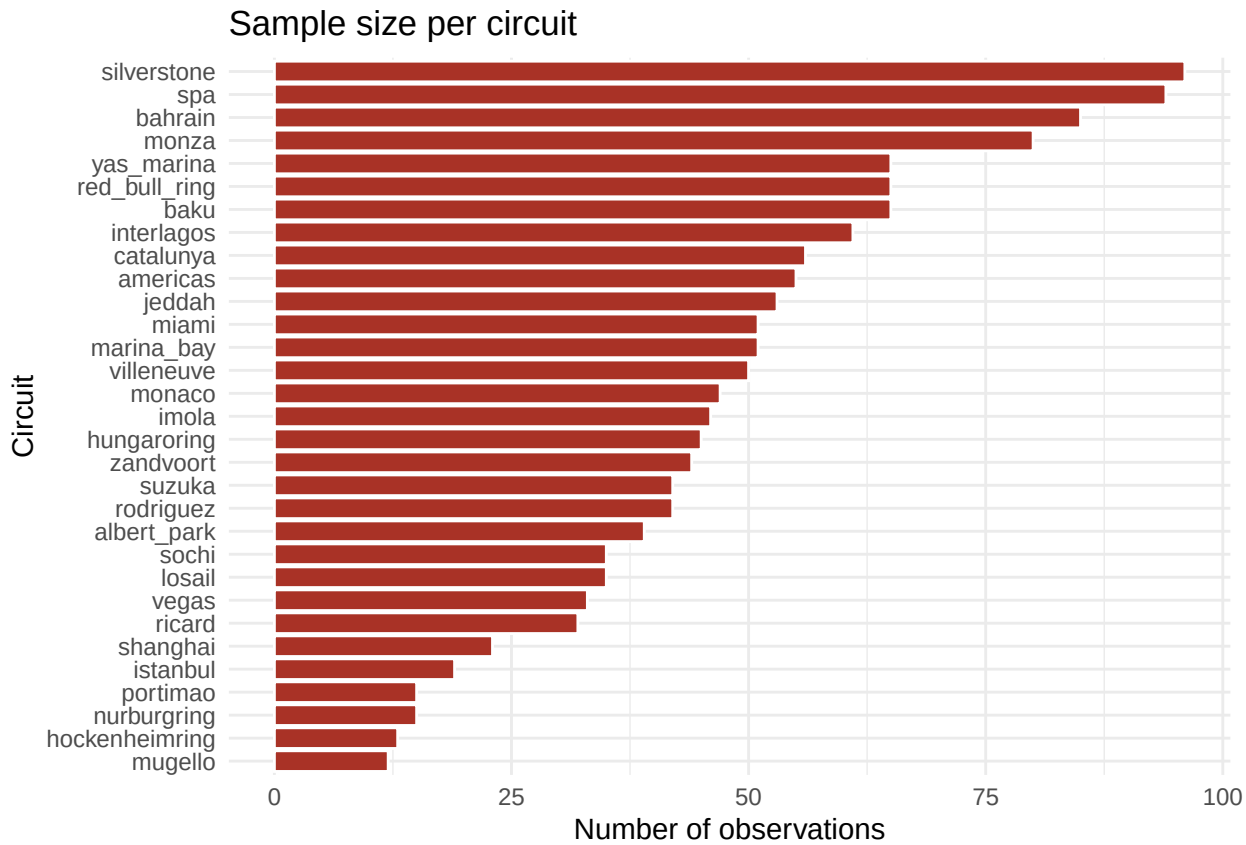
```r
head(data)
```

```
##         driver_id constructor_id points position grid laps   status    gap
## 1          bottas       mercedes     26        1    2   58 Finished  0.000
## 2        hamilton       mercedes     18        2    1   58 Finished 20.886
## 3  max_verstappen       red_bull     15        3    4   58 Finished 22.520
## 4          vettel        ferrari     12        4    3   58 Finished 57.109
## 5         leclerc        ferrari     10        5    5   58 Finished 58.203
## 6 kevin_magnussen           haas      8        6    7   58 Finished 87.156
```

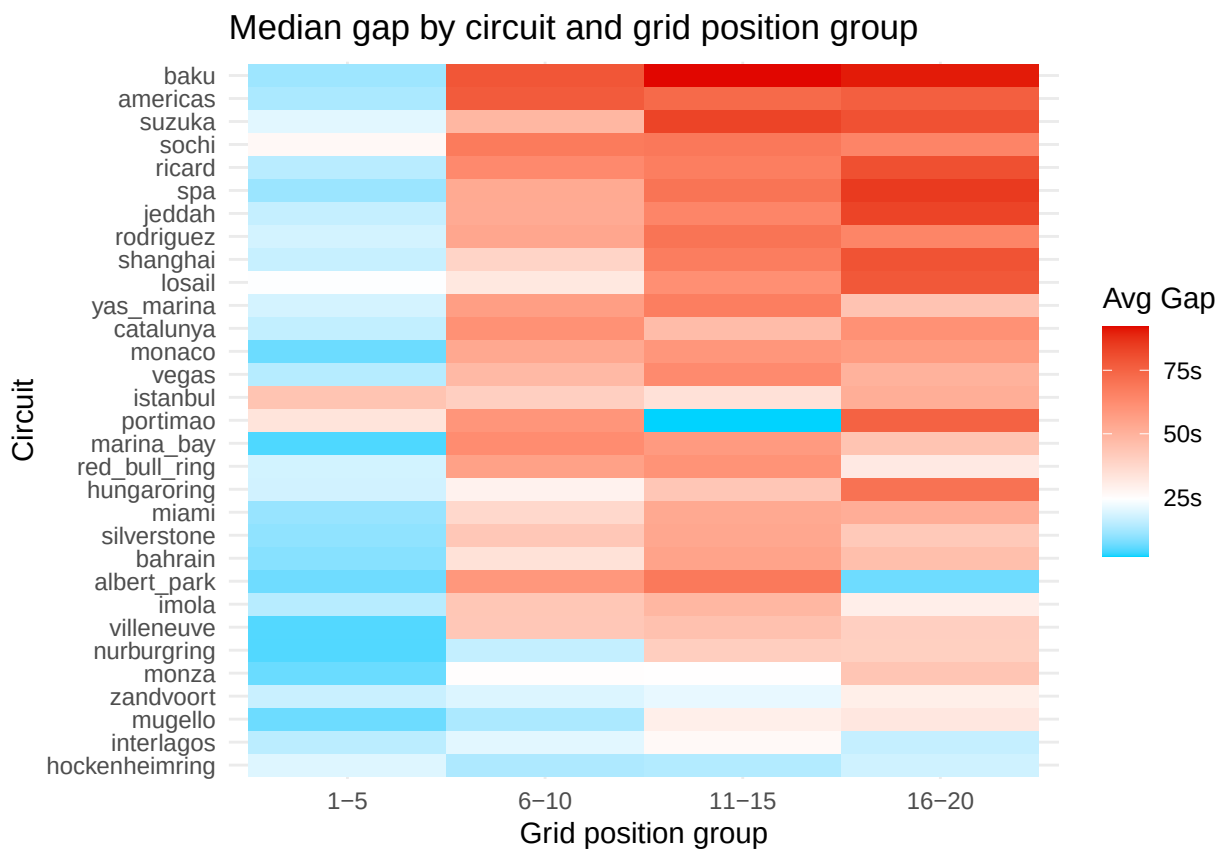
```
##   fastest_rank  fastest top_speed_kph time_sec season round
## 1            1 1:25.580       223.075   85.580   2019     1
## 2            2 1:26.057       221.839   86.057   2019     1
## 3            3 1:26.256       221.327   86.256   2019     1
## 4            8 1:27.954       217.054   87.954   2019     1
## 5            4 1:26.926       219.621   86.926   2019     1
## 6            9 1:28.182       216.493   88.182   2019     1
##               race_name     circuit circuit_id
## 1 Australian Grand Prix albert_park          1
## 2 Australian Grand Prix albert_park          1
## 3 Australian Grand Prix albert_park          1
## 4 Australian Grand Prix albert_park          1
## 5 Australian Grand Prix albert_park          1
## 6 Australian Grand Prix albert_park          1
```

```r
# data |> distinct(circuit, circuit_id) |> arrange(circuit_id)
```

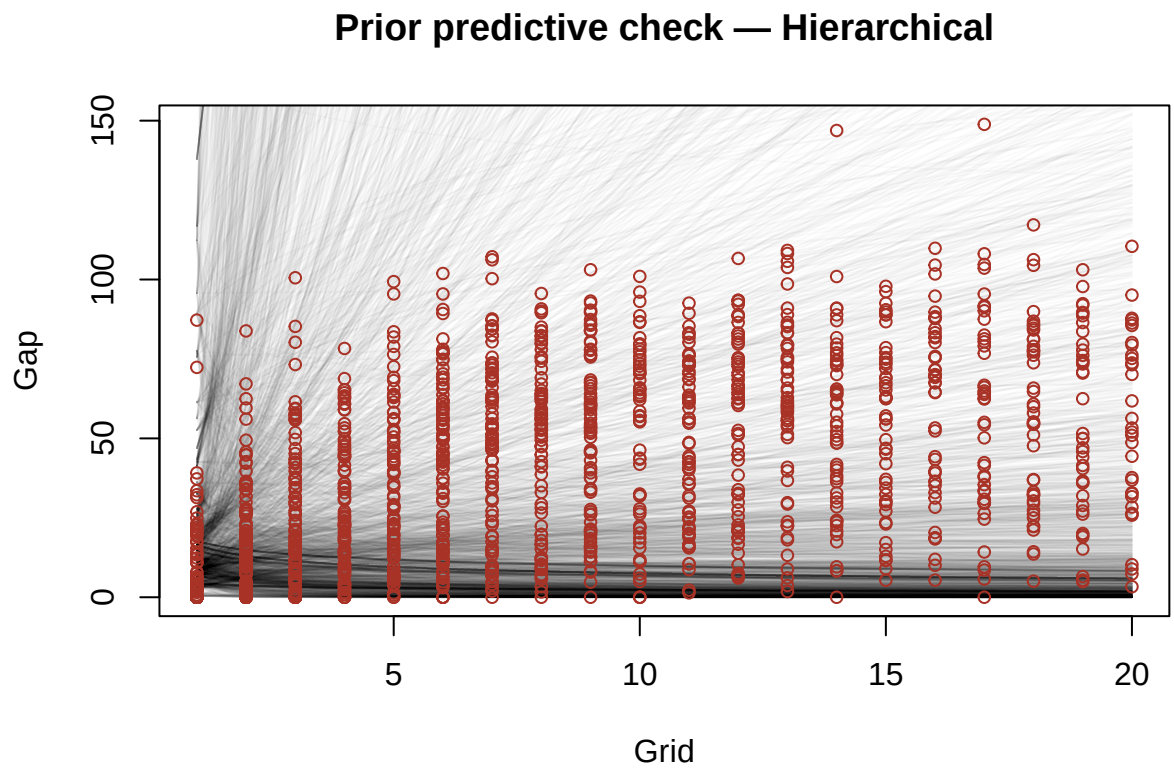
```r
# Sample size per circuit

data |>
  count(circuit) |>
  ggplot(aes(x = reorder(circuit, n), y = n)) +
  geom_bar(stat = "identity", fill = "#a93226", color = "white") +
  coord_flip() +
  theme_minimal() +
  labs(
    title = "Sample size per circuit",
    x     = "Circuit",
    y     = "Number of observations"
  )
```




```r
data |>
  mutate(grid_group = cut(grid, breaks = c(0, 5, 10, 15, 20),
                          labels = c("1-5", "6-10", "11-15", "16-20"))) |>
  group_by(circuit, grid_group) |>
  summarise(median_gap = median(gap), .groups = "drop") |>
  ggplot(aes(x = grid_group, y = reorder(circuit, median_gap), fill = median_gap)) +
  geom_tile() +
  scale_fill_gradientn(
    colors = c("#00d3ff", "#ffffff", "#e10600"),
    values = scales::rescale(c(0, 30, 120)),  # adjust midpoint if needed
    labels = scales::label_number(suffix = "s"),
    name = "Avg Gap"
  ) +
  theme_minimal() +
  labs(
    title = "Median gap by circuit and grid position group",
    x     = "Grid position group",
    y     = "Circuit",
    fill  = "Median gap (s)"
  )
```




```r
# Prior predictive check for hierarchical model showing circuit-level variation
n_samples  <- 100
xs         <- seq(1, 20, by = 0.1)

alpha_prior      <- rnorm(n_samples, -2.5, 1.0)
mu_beta_prior    <- rnorm(n_samples, -0.5, 0.5)
sigma_beta_prior <- rexp(n_samples, 1)

plot(NULL, xlim = c(1, 20), ylim = range(data$gap),
     xlab = "Grid", ylab = "Gap",
     main = "Prior predictive check - Hierarchical")

for (i in 1:n_samples) {
  # sample circuit-specific beta from hierarchical prior
  beta_j_i <- rnorm(N_circuits, mu_beta_prior[i], sigma_beta_prior[i])
  # plot one curve per circuit
  for (j in 1:N_circuits) {
    mean_gap <- exp(-(alpha_prior[i] + beta_j_i[j] * log(xs)))
    lines(xs, mean_gap, col = rgb(0, 0, 0, alpha = 0.02))
  }
}

points(data$grid, data$gap, col = "#a93226", pch = 1, cex = 0.8)
```




```r
# hierarchical Stan model and fit using MCMC with 5 chains

dir.create(file.path("stan_out_hierarchical"), showWarnings = FALSE)

grid_pred_pos    <- 10
circuit_pred_pos <- 1

mod_hierarchical <- cmdstan_model("hierarchical_model_exp.stan")

fit_hierarchical <- mod_hierarchical$sample(
  seed          = 1,
  chains        = 5,
  refresh       = 0,
  output_dir    = "stan_out_hierarchical",
  data          = list(
    N            = N_obs,
    J            = N_circuits,
    circuit_id   = data$circuit_id,
    grid         = data$grid,
    gap          = data$gap,
    grid_pred    = grid_pred_pos,
    circuit_pred = circuit_pred_pos
  ),
  iter_warmup   = 500,
  iter_sampling = 500
)
```

```r
# print(fit_hierarchical)
```

```r
# hierarchical model using variational inference

fit_hierarchical_vi <- mod_hierarchical$variational(
  seed             = 1,
  refresh          = 0,
  show_messages    = FALSE,
  show_exceptions  = FALSE,
  output_dir       = "stan_out_hierarchical",
  algorithm        = "fullrank",
  output_samples   = 1000,
  data             = list(
    N            = N_obs,
    J            = N_circuits,
    circuit_id   = data$circuit_id,
    grid         = data$grid,
    gap          = data$gap,
    grid_pred    = grid_pred_pos,
    circuit_pred = circuit_pred_pos
  )
)
```

```r
# ========= model summary (ESS and R hat) ================
print(fit_hierarchical)
```

```
##    variable     mean   median   sd  mad       q5      q95 rhat ess_bulk
##  lp__       -6574.57 -6574.44 5.52 5.38 -6583.77 -6565.55 1.01      457
##  alpha         -2.25    -2.25 0.07 0.07    -2.36    -2.14 1.01      485
##  mu_beta       -0.70    -0.70 0.04 0.04    -0.76    -0.63 1.00      499
##  sigma_beta     0.09     0.09 0.02 0.02     0.05     0.13 1.00      604
##  beta_j[1]     -0.69    -0.69 0.07 0.07    -0.81    -0.58 1.00     1148
##  beta_j[2]     -0.77    -0.77 0.06 0.06    -0.87    -0.67 1.00     1188
##  beta_j[3]     -0.65    -0.65 0.06 0.06    -0.74    -0.56 1.00     1028
##  beta_j[4]     -0.77    -0.77 0.06 0.06    -0.87    -0.68 1.00      849
##  beta_j[5]     -0.74    -0.74 0.06 0.06    -0.85    -0.63 1.00     1212
##  beta_j[6]     -0.58    -0.59 0.10 0.10    -0.73    -0.40 1.00     1364
##  ess_tail
##       825
##       739
##       854
##       972
##      1338
##      1562
##      1446
##      1344
##      1721
##      1899
##
##  # showing 10 of 36 rows (change via 'max_rows' argument or 'cmdstanr_max_rows' option)
```

```r
fit_hierarchical$summary(NULL, c("mean", "sd", "mcse_mean", "ess_bulk"))
```

```
## # A tibble: 36 x 5
##    variable         mean     sd mcse_mean ess_bulk
```

```
##    <chr>           <dbl>  <dbl>     <dbl>    <dbl>
##  1 lp__        -6575.     5.52    0.257        458.
##  2 alpha          -2.25   0.0700  0.00318      486.
##  3 mu_beta        -0.696  0.0393  0.00176      499.
##  4 sigma_beta      0.0912 0.0241  0.000951     605.
##  5 beta_j[1]      -0.692  0.0689  0.00204     1149.
##  6 beta_j[2]      -0.768  0.0620  0.00180     1188.
##  7 beta_j[3]      -0.651  0.0560  0.00174     1029.
##  8 beta_j[4]      -0.773  0.0592  0.00202      850.
##  9 beta_j[5]      -0.740  0.0646  0.00185     1213.
## 10 beta_j[6]      -0.581  0.0995  0.00268     1364.
## # i 26 more rows
```

```r
print(fit_hierarchical_vi)
```

```
##     variable     mean   median    sd  mad       q5      q95
##  lp__        -6626.57 -6625.39 10.52 9.96 -6644.66 -6611.41
##  lp_approx__   -17.08   -16.90  4.19 4.35   -24.26   -10.51
##  alpha          -2.06    -2.06  0.13 0.13    -2.27    -1.83
##  mu_beta        -0.80    -0.80  0.07 0.07    -0.92    -0.68
##  sigma_beta      0.15     0.14  0.03 0.03     0.11     0.19
##  beta_j[1]      -0.79    -0.79  0.12 0.12    -0.99    -0.59
##  beta_j[2]      -0.89    -0.89  0.09 0.09    -1.04    -0.73
##  beta_j[3]      -0.73    -0.73  0.09 0.09    -0.88    -0.58
##  beta_j[4]      -0.89    -0.89  0.09 0.08    -1.03    -0.75
##  beta_j[5]      -0.86    -0.86  0.11 0.11    -1.05    -0.68
##
##  # showing 10 of 37 rows (change via 'max_rows' argument or 'cmdstanr_max_rows' option)
```

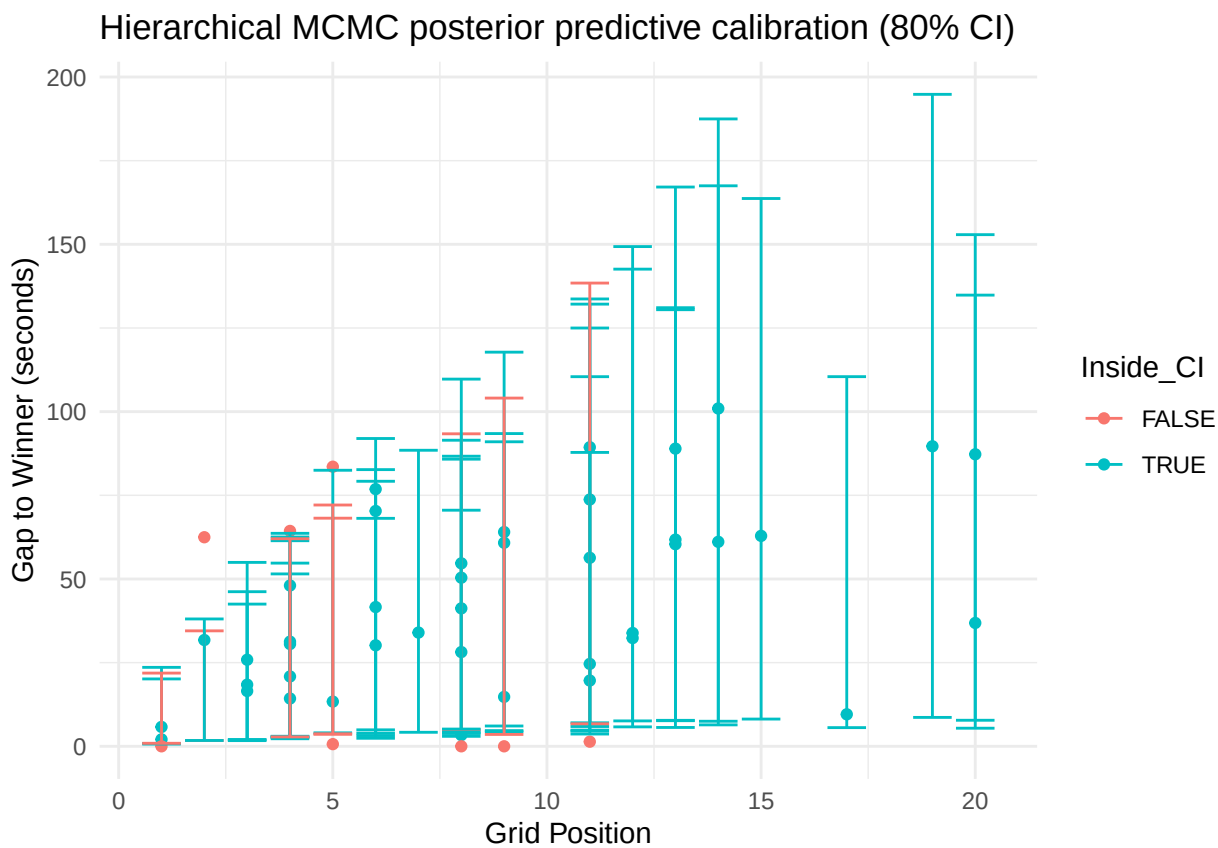
```r
fit_hierarchical_vi$summary(NULL, c("mean", "sd", "mcse_mean", "ess_bulk"))
```

```
## # A tibble: 37 x 5
##    variable         mean      sd mcse_mean ess_bulk
##    <chr>           <dbl>   <dbl>     <dbl>    <dbl>
##  1 lp__        -6627.    10.5     0.323       1053.
##  2 lp_approx__   -17.1    4.19    0.139        924.
##  3 alpha          -2.06   0.129   0.00404     1022.
##  4 mu_beta        -0.803  0.0716  0.00226      999.
##  5 sigma_beta      0.145  0.0258  0.000769    1129.
##  6 beta_j[1]      -0.794  0.119   0.00383      973.
##  7 beta_j[2]      -0.887  0.0938  0.00293     1022.
##  8 beta_j[3]      -0.729  0.0923  0.00311      877.
##  9 beta_j[4]      -0.890  0.0853  0.00291      867.
## 10 beta_j[5]      -0.865  0.109   0.00326     1127.
## # i 27 more rows
```

```r
# Setup calibration parameters and sample validation observations
set.seed(1)
N_train = N_obs - 1

ci_level = 0.8
ci_plims = c((1 - ci_level) / 2, (1 + ci_level) / 2)

validation_obs <- sample(1:N_obs, 50, replace = FALSE)
```

```r
# MCMC leave-one-out calibration loop over 50 validation observations
all_mcmc_samples <- list()
ci_limits_hier = matrix(0, nrow = 50, 2)

for (i in 1:length(validation_obs)) {
  i_test = validation_obs[i]
  train_test_dta = list(
    N            = N_train,
    J            = N_circuits,
    circuit_id   = data$circuit_id[-i_test],
    grid         = data$grid[-i_test],
    gap          = data$gap[-i_test],
    grid_pred    = data$grid[i_test],
    circuit_pred = data$circuit_id[i_test]
  )
  mcmc_results = mod_hierarchical$sample(
    seed             = 1,
    data             = train_test_dta,
    output_dir       = "stan_out_hierarchical",
    refresh          = 0,
    show_messages    = FALSE,
    show_exceptions  = FALSE,
    chains           = 1,
    iter_warmup      = 500,
    iter_sampling    = 500
  )
  hier_samples           = as.vector(mcmc_results$draws("gap_pred"))
  all_mcmc_samples[[i]] = as.vector(mcmc_results$draws("gap_pred"))
  ci_limits_hier[i, ]   = quantile(hier_samples, ci_plims)
}
```

```r
# MCMC calibration plot showing 80% credible intervals vs observed gaps
merged_data_hier = data[validation_obs, ] |>
  bind_cols(data.frame(CI_L = ci_limits_hier[, 1], CI_R = ci_limits_hier[, 2])) |>
  mutate(Inside_CI = (gap >= CI_L & gap <= CI_R))

merged_data_hier |>
  ggplot(aes(x = grid, y = gap, ymin = CI_L, ymax = CI_R, color = Inside_CI)) +
  geom_point() +
  geom_errorbar() +
  theme_minimal() +
  labs(title = "Hierarchical MCMC posterior predictive calibration (80% CI)",
       x = "Grid Position", y = "Gap to Winner (seconds)")
```



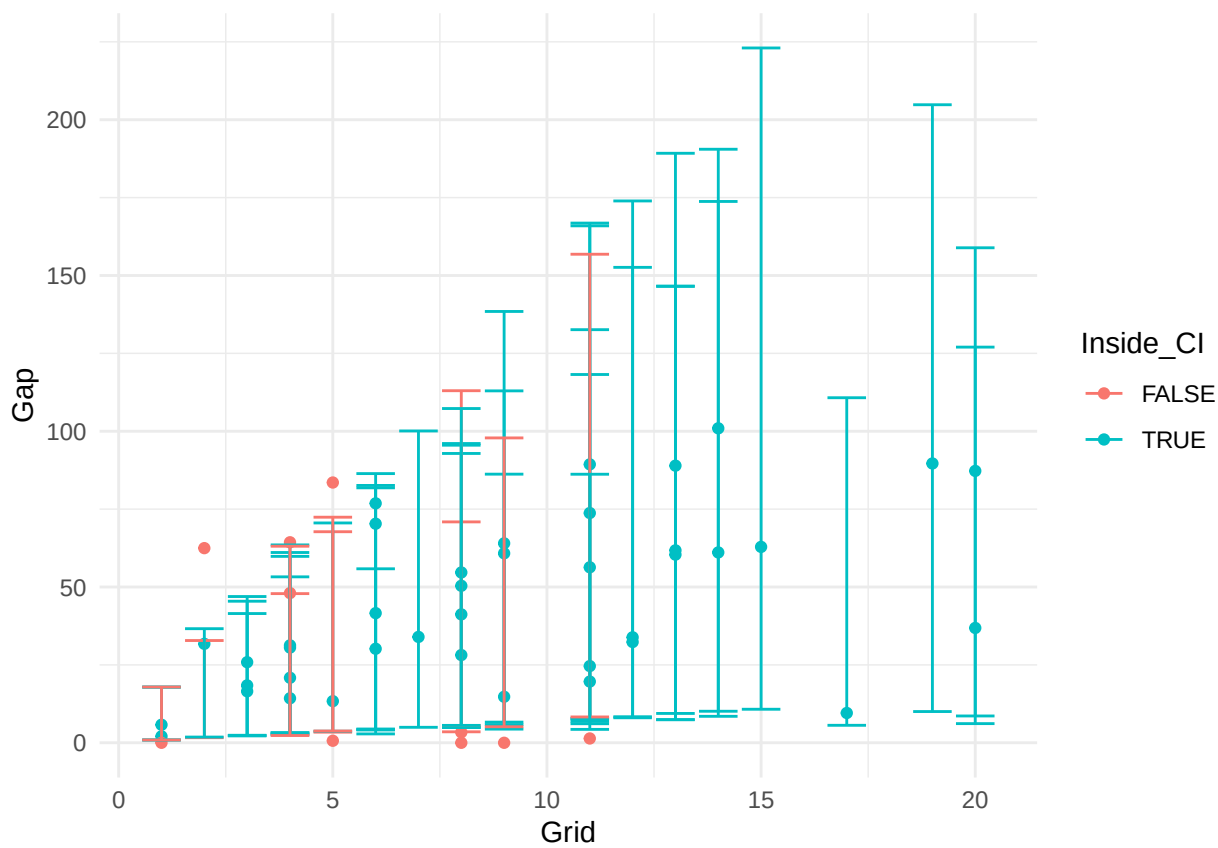

```r
mean(merged_data_hier$Inside_CI)
```

```
## [1] 0.84
```

```r
# VI leave-one-out calibration loop over 50 validation observations with clipping
all_vi_samples <- list()
ci_limits_hier_var = matrix(0, nrow = 50, 2)

for (i in 1:length(validation_obs)) {
  i_test = validation_obs[i]
  train_test_dta = list(
    N            = N_train,
    J            = N_circuits,
    circuit_id   = data$circuit_id[-i_test],
    grid         = data$grid[-i_test],
    gap          = data$gap[-i_test],
    grid_pred    = data$grid[i_test],
    circuit_pred = data$circuit_id[i_test]
  )
  vi_results = mod_hierarchical$variational(
    seed           = 1,
    data           = train_test_dta,
    output_dir     = "stan_out_hierarchical",
    refresh        = 0,
    show_messages  = FALSE,
    show_exceptions = FALSE,
    algorithm      = "fullrank",
```

```r
    output_samples = 1000
  )
  hier_var_samples        = as.vector(vi_results$draws("gap_pred"))
  # hier_var_samples        = pmin(hier_var_samples, 500)
  all_vi_samples[[i]]     = hier_var_samples
  ci_limits_hier_var[i, ] = quantile(hier_var_samples, ci_plims)
}
```

```r
# VI calibration plot showing 80% credible intervals vs observed gaps
merged_data_hier_var = data[validation_obs, ] |>
  bind_cols(data.frame(CI_L = ci_limits_hier_var[, 1], CI_R = ci_limits_hier_var[, 2])) |>
  mutate(Inside_CI = (gap >= CI_L & gap <= CI_R))

merged_data_hier_var |>
  ggplot(aes(x = grid, y = gap, ymin = CI_L, ymax = CI_R, color = Inside_CI)) +
  geom_point() +
  geom_errorbar() +
  theme_minimal() +
  labs(x = "Grid", y = "Gap")
```



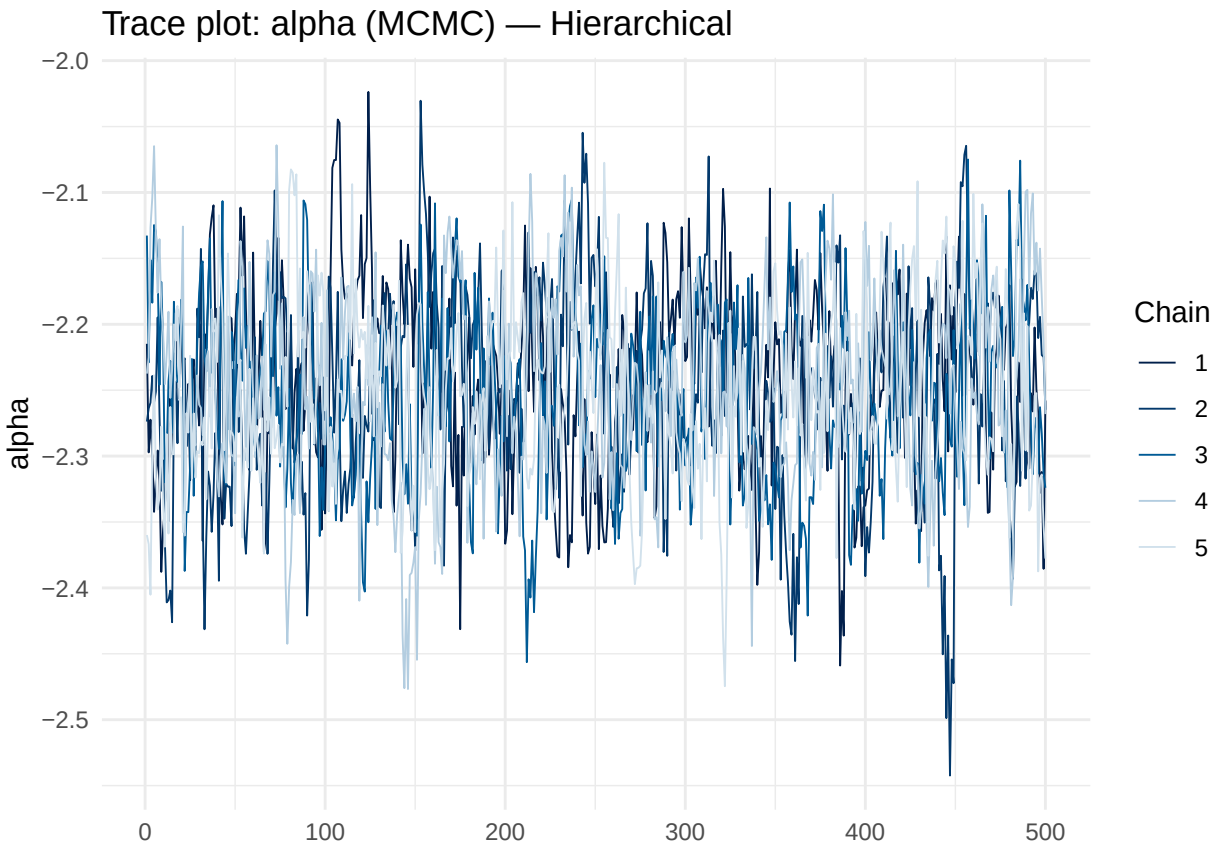

```r
mean(merged_data_hier_var$Inside_CI)
```

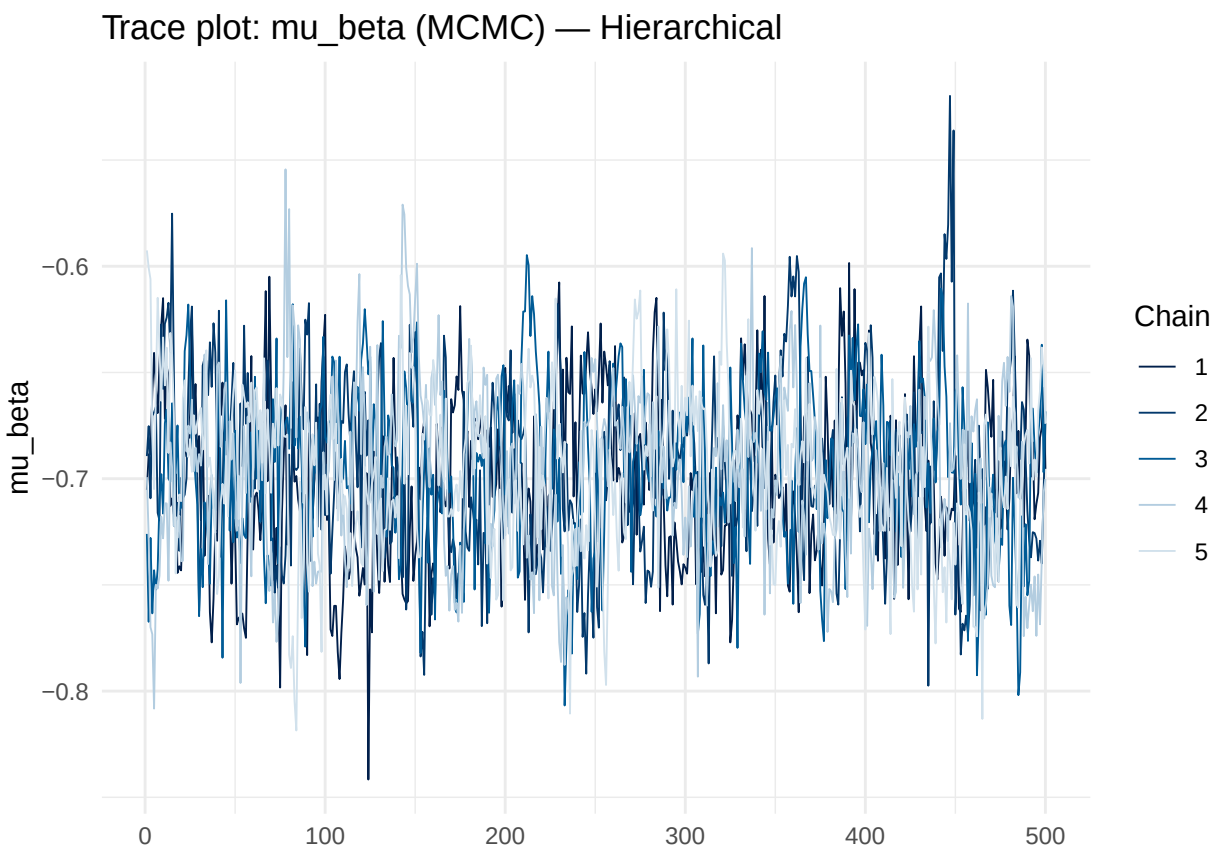
```
## [1] 0.8
```

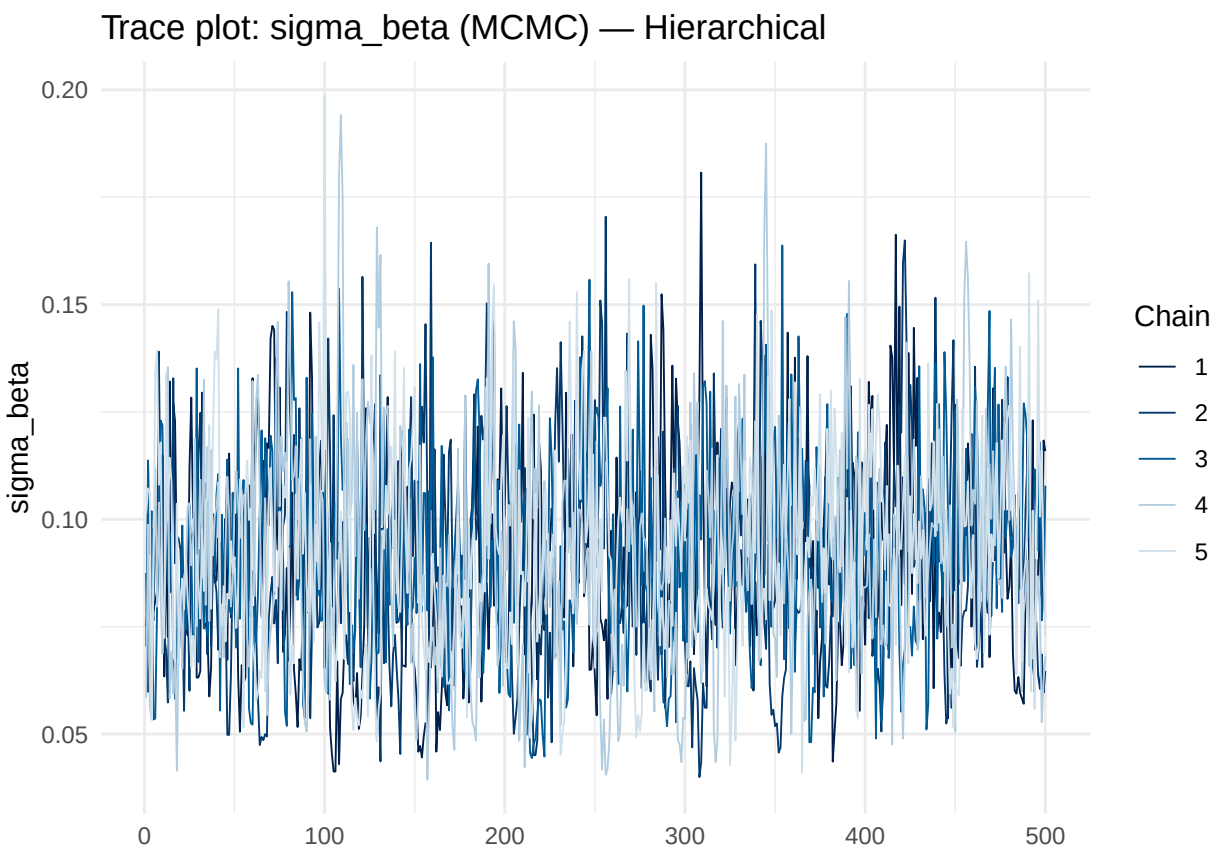
```r
print("end")
```

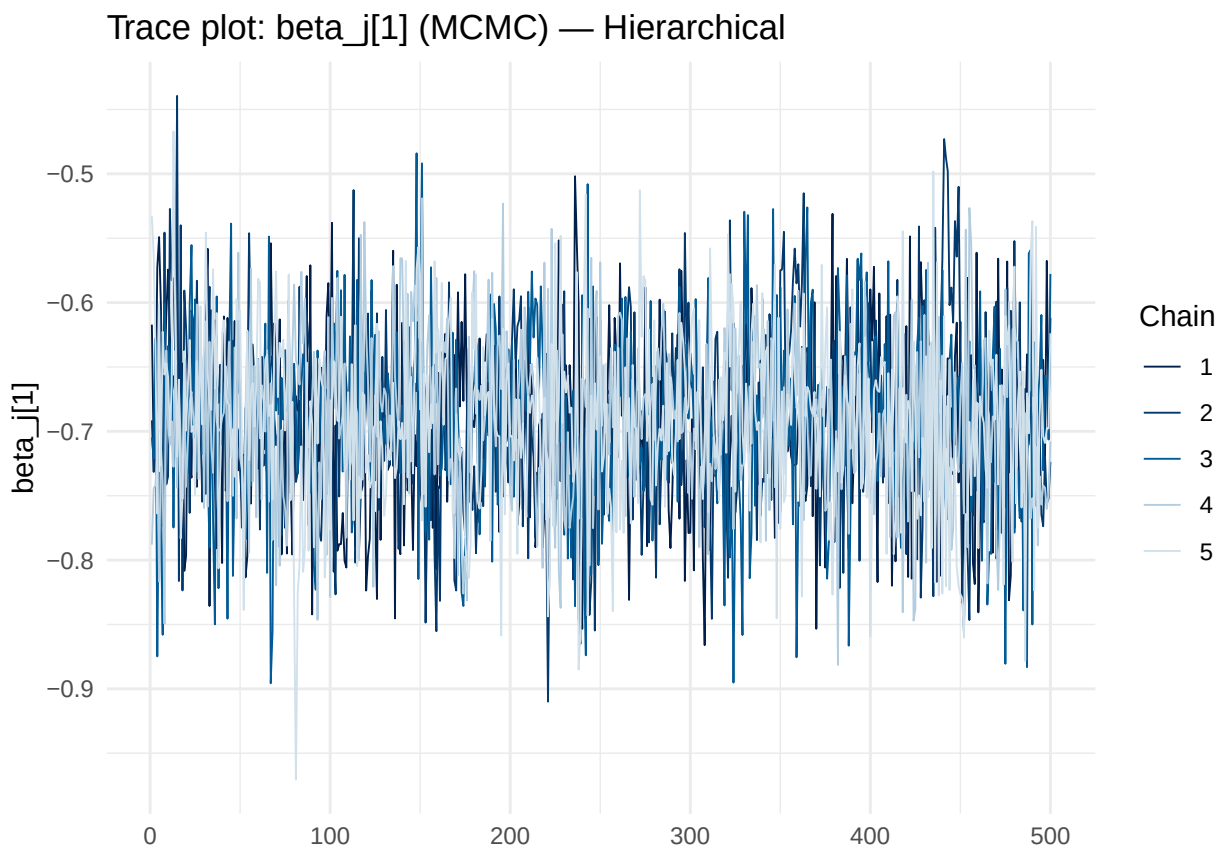
```
## [1] "end"
```

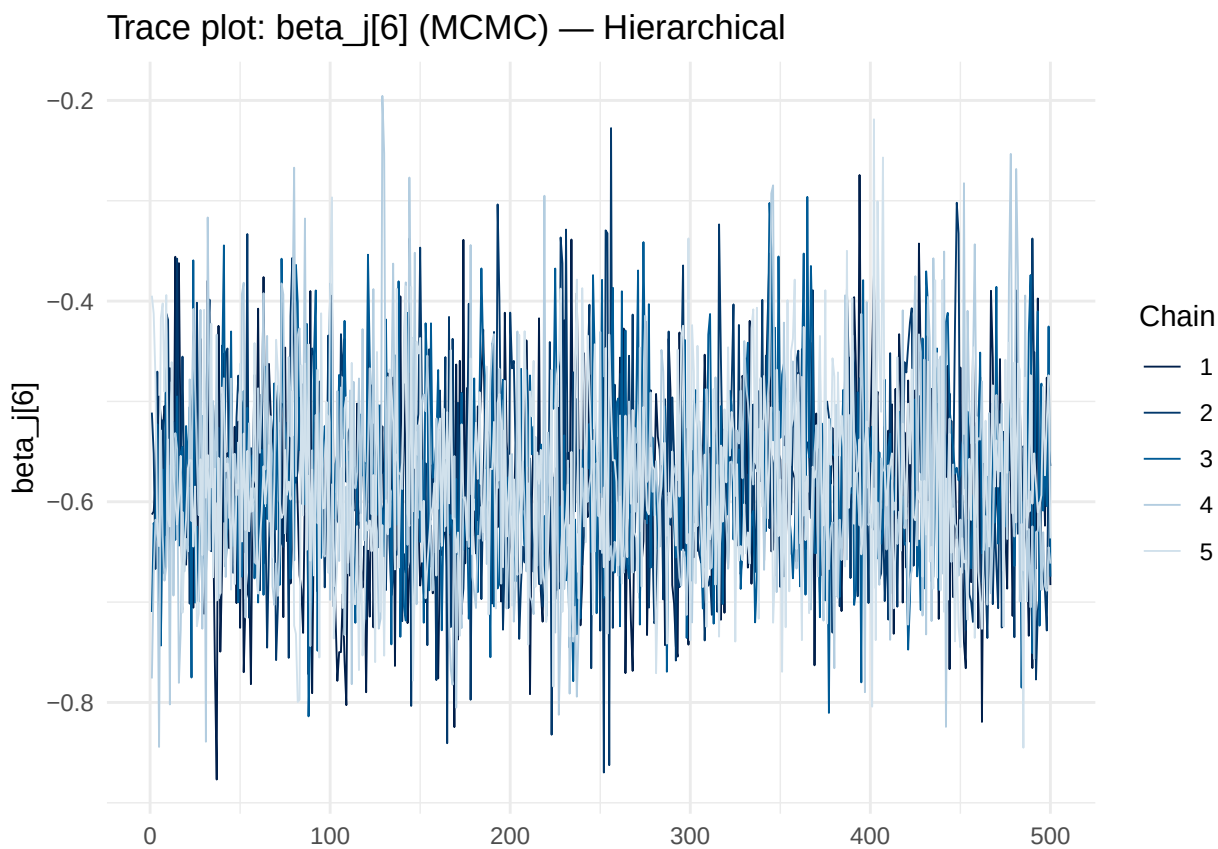
```r
# Trace plots for global and circuit-level parameters to assess MCMC mixing
dir.create("plots/trace_plots_Hierarchical", recursive = TRUE, showWarnings = FALSE)

p1 <- mcmc_trace(fit_hierarchical$draws("alpha")) +
  theme_minimal() +
  labs(title = "Trace plot: alpha (MCMC) - Hierarchical")
ggsave("plots/trace_plots_Hierarchical/trace_mcmc_Hierarchical_alpha.png", p1, width = 8, height = 4)

p2 <- mcmc_trace(fit_hierarchical$draws("mu_beta")) +
  theme_minimal() +
  labs(title = "Trace plot: mu_beta (MCMC) - Hierarchical")
ggsave("plots/trace_plots_Hierarchical/trace_mcmc_Hierarchical_mu_beta.png", p2, width = 8, height = 4)

p3 <- mcmc_trace(fit_hierarchical$draws("sigma_beta")) +
  theme_minimal() +
  labs(title = "Trace plot: sigma_beta (MCMC) - Hierarchical")
ggsave("plots/trace_plots_Hierarchical/trace_mcmc_Hierarchical_sigma_beta.png", p3, width = 8, height =

p4 <- mcmc_trace(fit_hierarchical$draws("beta_j[1]")) +
  theme_minimal() +
  labs(title = "Trace plot: beta_j[1] (MCMC) - Hierarchical")
ggsave("plots/trace_plots_Hierarchical/trace_mcmc_Hierarchical_beta_j1.png", p4, width = 8, height = 4)

p5 <- mcmc_trace(fit_hierarchical$draws("beta_j[6]")) +
  theme_minimal() +
  labs(title = "Trace plot: beta_j[6] (MCMC) - Hierarchical")
  ggsave("plots/trace_plots_Hierarchical/trace_mcmc_Hierarchical_beta_j6.png", p5, width = 8, height = 4

p1; p2; p3; p4; p5
```


Trace plot: alpha (MCMC) — Hierarchical
alpha
−2.0
−2.1
−2.2
−2.3
−2.4
−2.5
0
100
200
300
400
500
Chain
1
2
3
4
5


Trace plot: mu_beta (MCMC) — Hierarchical
mu_beta
−0.6
−0.7
−0.8
0
100
200
300
400
500
Chain
1
2
3
4
5

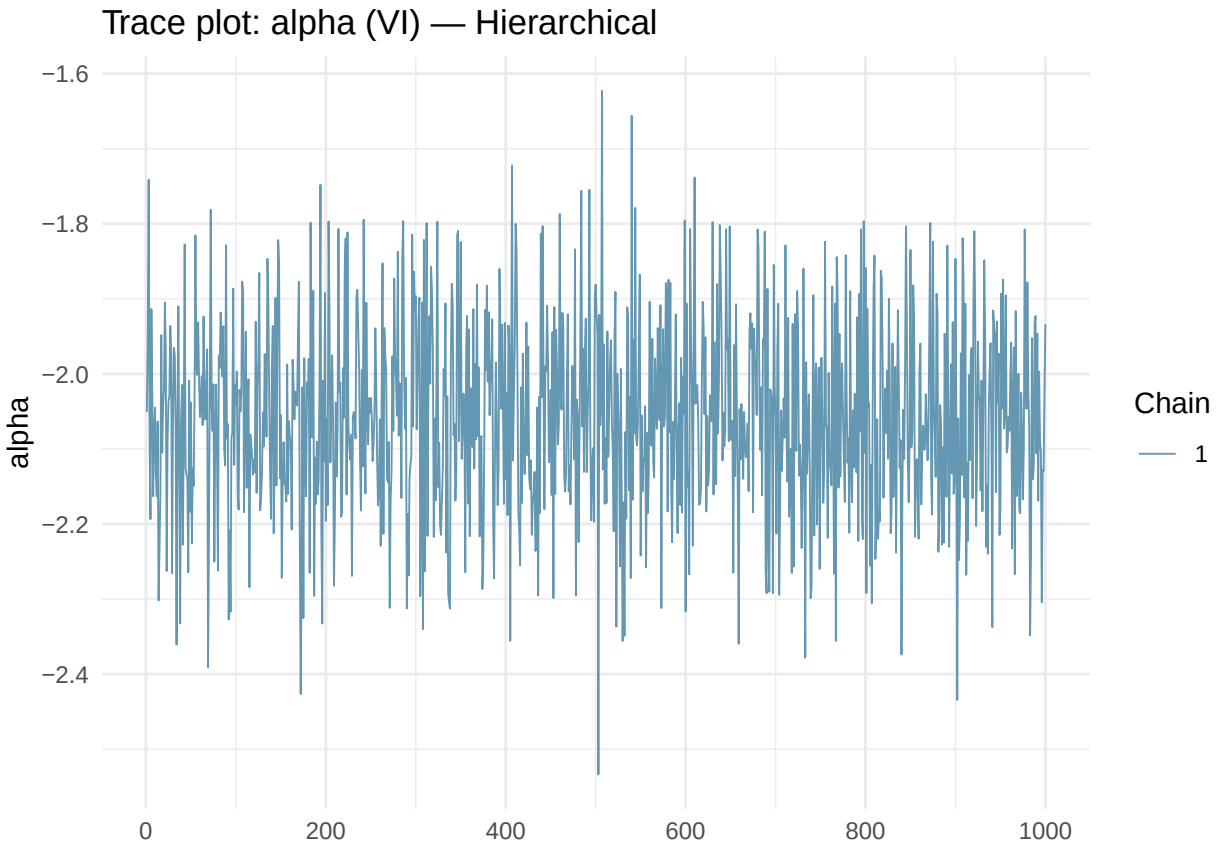

Trace plot: sigma_beta (MCMC) — Hierarchical
sigma_beta
0.20
0.15
0.10
0.05
0
100
200
300
400
500
Chain
1
2
3
4
5

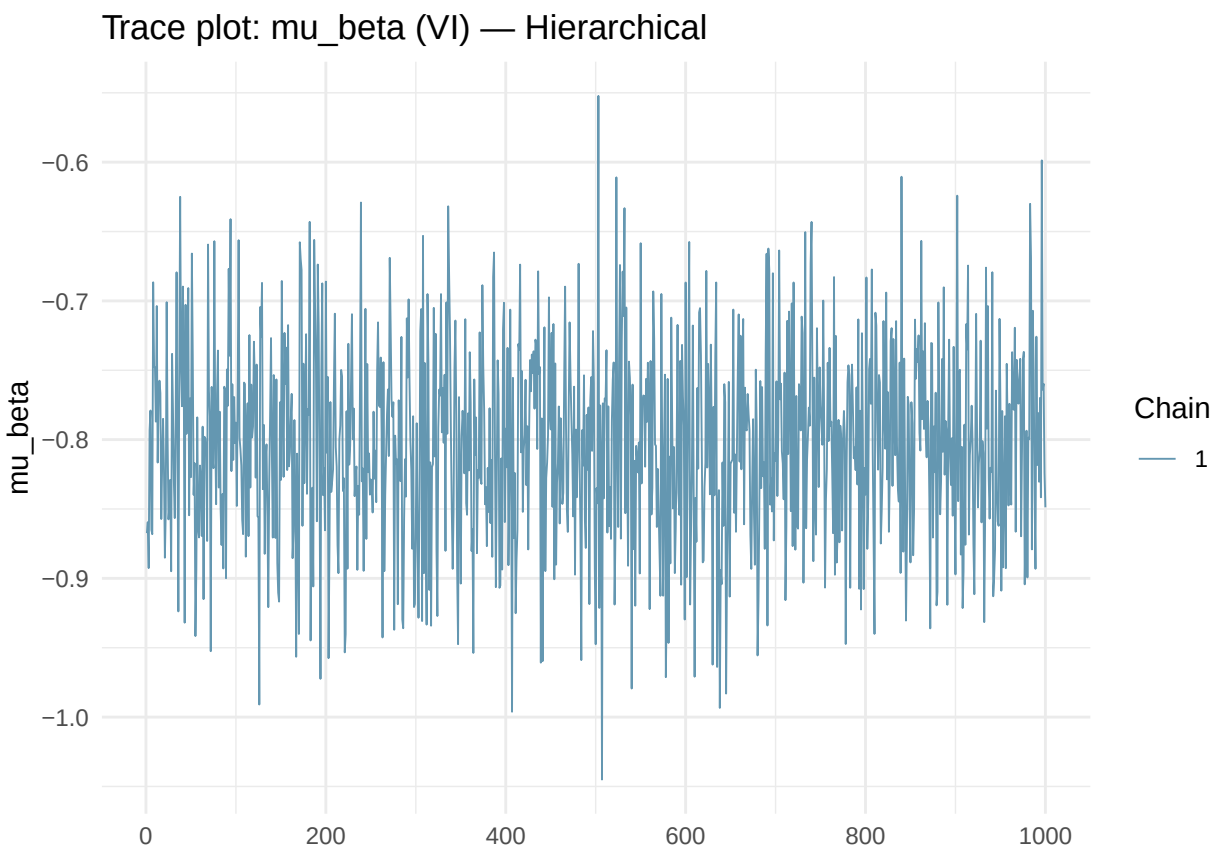

Trace plot: beta_j[1] (MCMC) — Hierarchical
beta_j[1]
−0.5
−0.6
−0.7
−0.8
−0.9
0
100
200
300
400
500
Chain
1
2
3
4
5

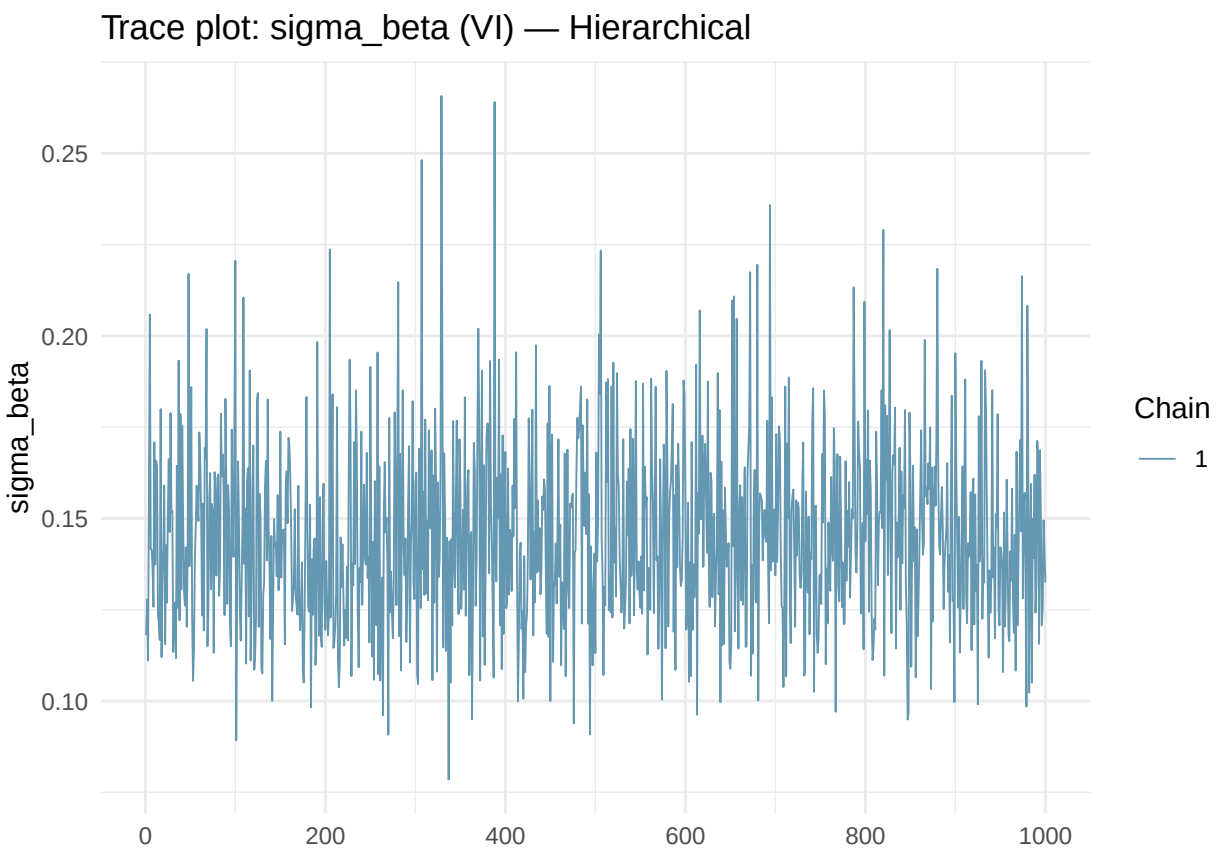


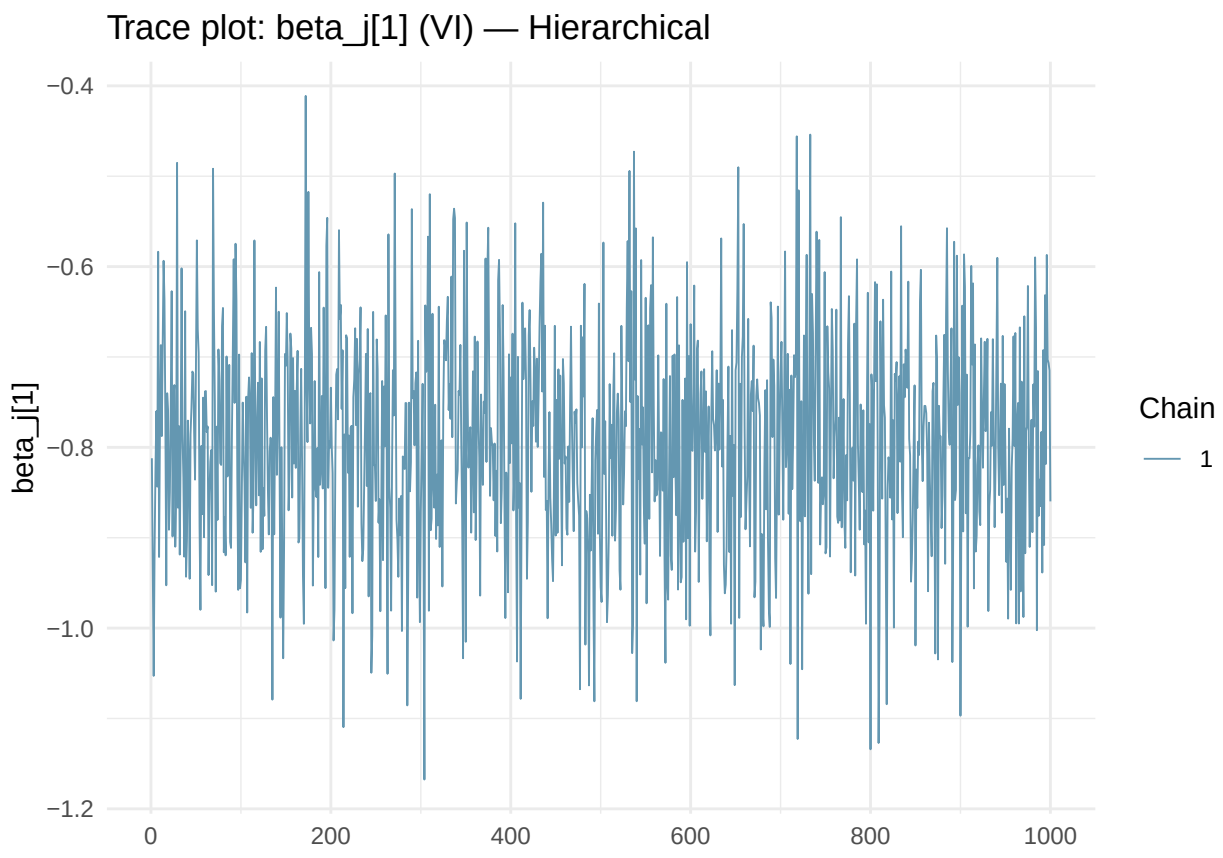

```r
# Trace plots for global and circuit-level parameters to assess VI mixing
p6 <- mcmc_trace(fit_hierarchical_vi$draws("alpha")) +
  theme_minimal() +
  labs(title = "Trace plot: alpha (VI) - Hierarchical")
ggsave("plots/trace_plots_Hierarchical/trace_vi_Hierarchical_alpha.png", p6, width = 8, height = 4)

p7 <- mcmc_trace(fit_hierarchical_vi$draws("mu_beta")) +
  theme_minimal() +
  labs(title = "Trace plot: mu_beta (VI) - Hierarchical")
ggsave("plots/trace_plots_Hierarchical/trace_vi_Hierarchical_mu_beta.png", p7, width = 8, height = 4)

p8 <- mcmc_trace(fit_hierarchical_vi$draws("sigma_beta")) +
  theme_minimal() +
  labs(title = "Trace plot: sigma_beta (VI) - Hierarchical")
ggsave("plots/trace_plots_Hierarchical/trace_vi_Hierarchical_sigma_beta.png", p8, width = 8, height = 4

p9 <- mcmc_trace(fit_hierarchical_vi$draws("beta_j[1]")) +
  theme_minimal() +
  labs(title = "Trace plot: beta_j[1] (VI) - Hierarchical")
ggsave("plots/trace_plots_Hierarchical/trace_vi_Hierarchical_beta_j1.png", p9, width = 8, height = 4)

p10 <- mcmc_trace(fit_hierarchical_vi$draws("beta_j[6]")) +
  theme_minimal() +
  labs(title = "Trace plot: beta_j[6] (VI) - Hierarchical")
ggsave("plots/trace_plots_Hierarchical/trace_vi_Hierarchical_beta_j6.png", p10, width = 8, height = 4)
```

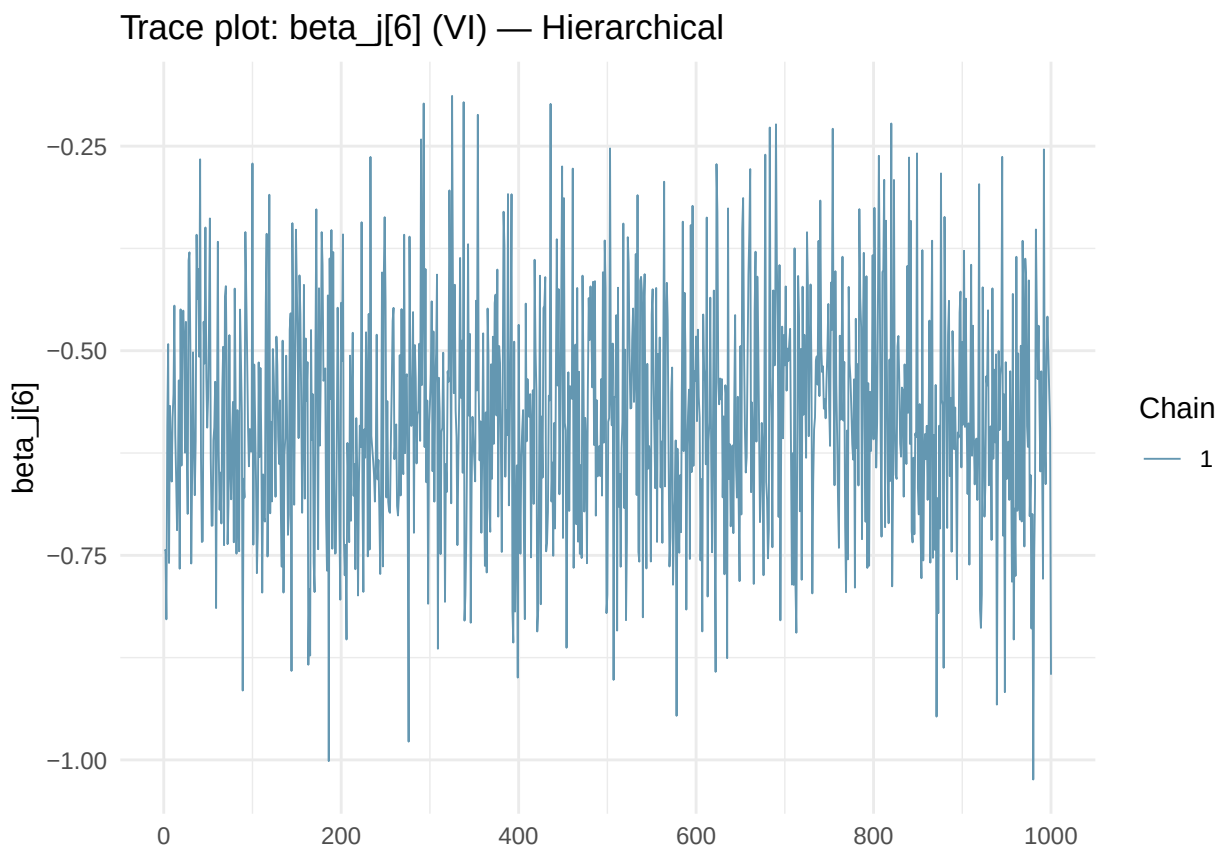
```
p6; p7; p8; p9; p10
```




Trace plot: mu_beta (VI) — Hierarchical
mu_beta
−0.6
−0.7
−0.8
−0.9
−1.0
0
200
400
600
800
1000
Chain
1

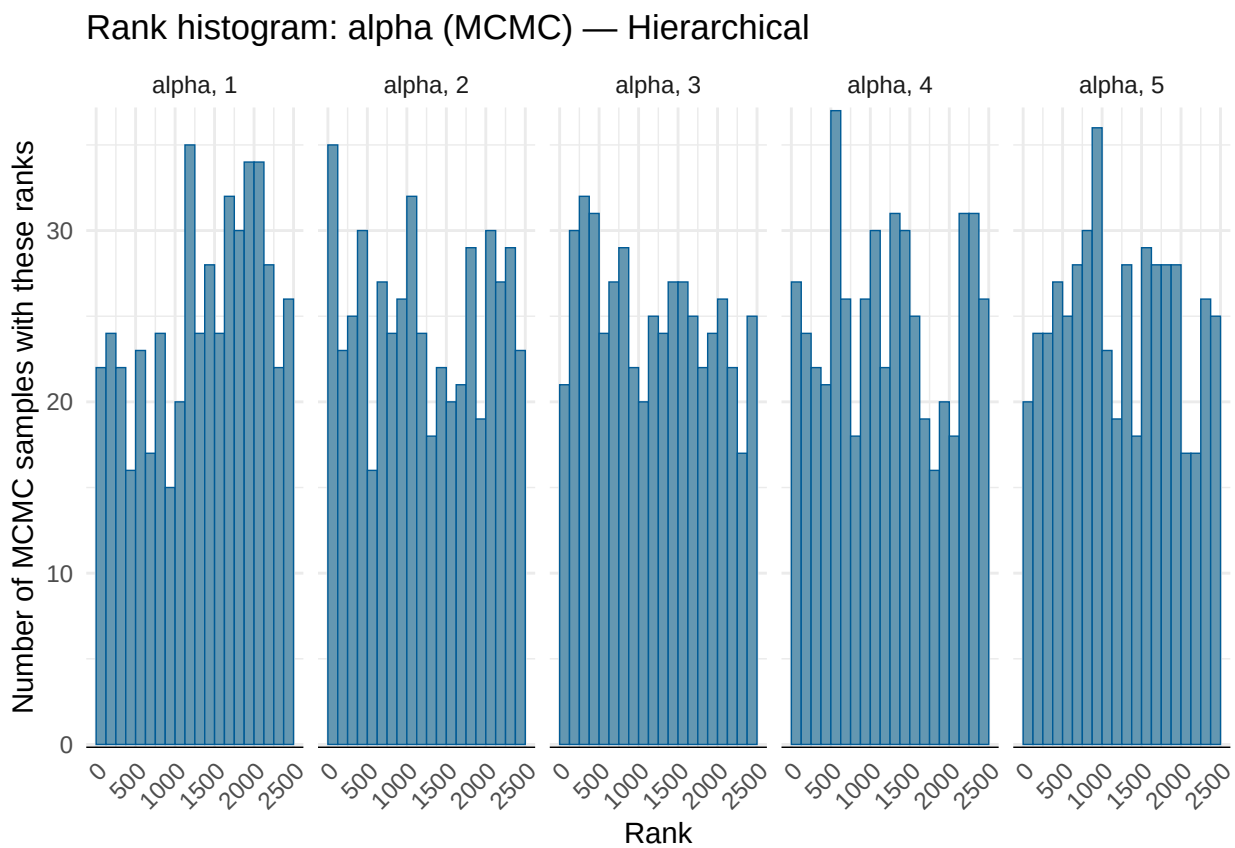

Trace plot: sigma_beta (VI) — Hierarchical
sigma_beta
0.25
0.20
0.15
0.10
0
200
400
600
800
1000
Chain
1

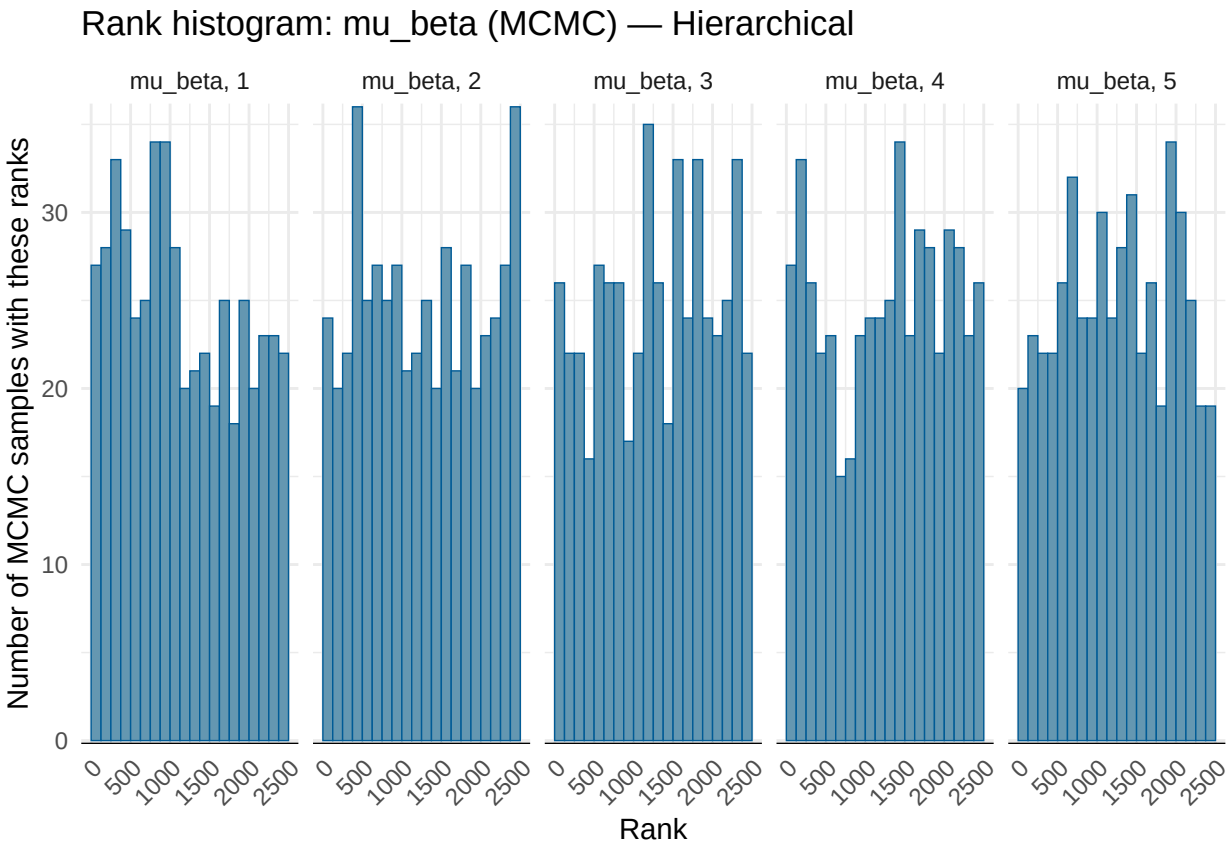

Trace plot: beta_j[1] (VI) — Hierarchical
beta_j[1]
−0.4
−0.6
−0.8
−1.0
−1.2
0
200
400
600
800
1000
Chain
1

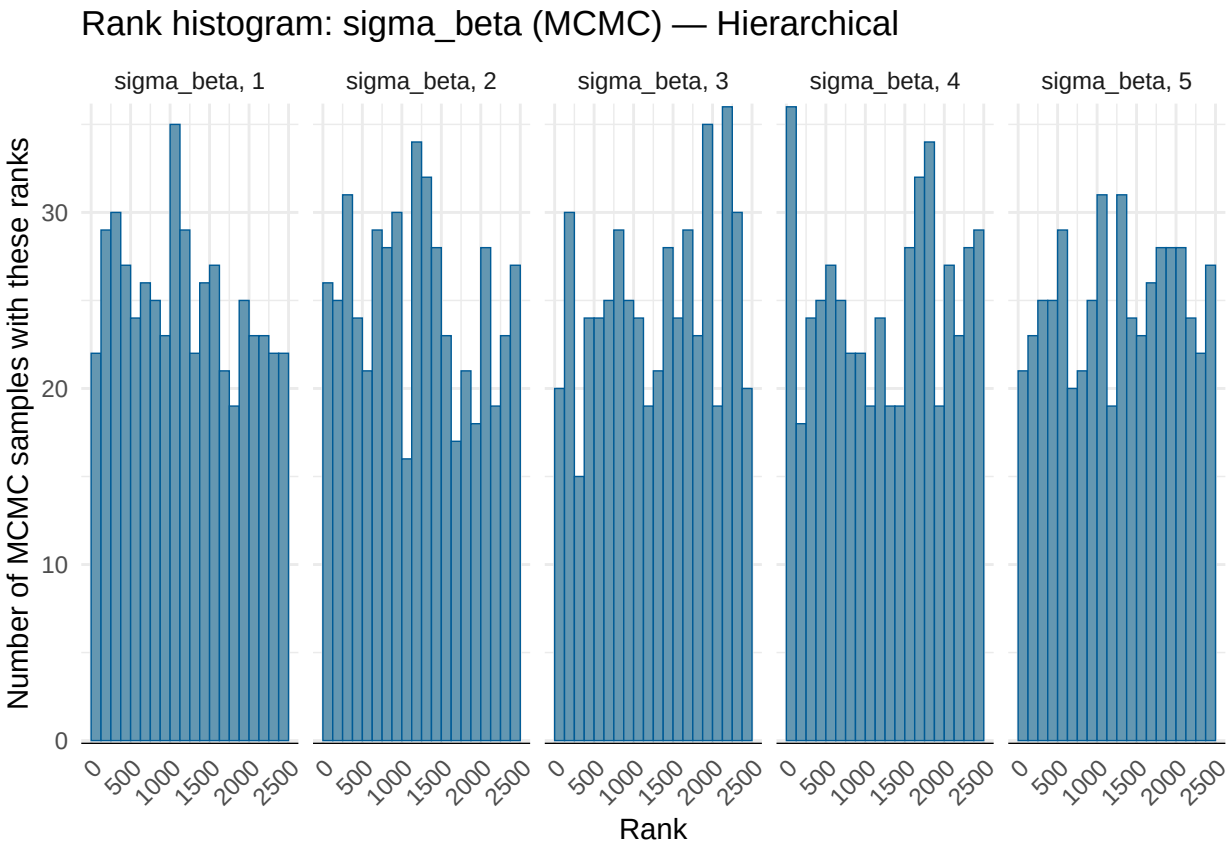


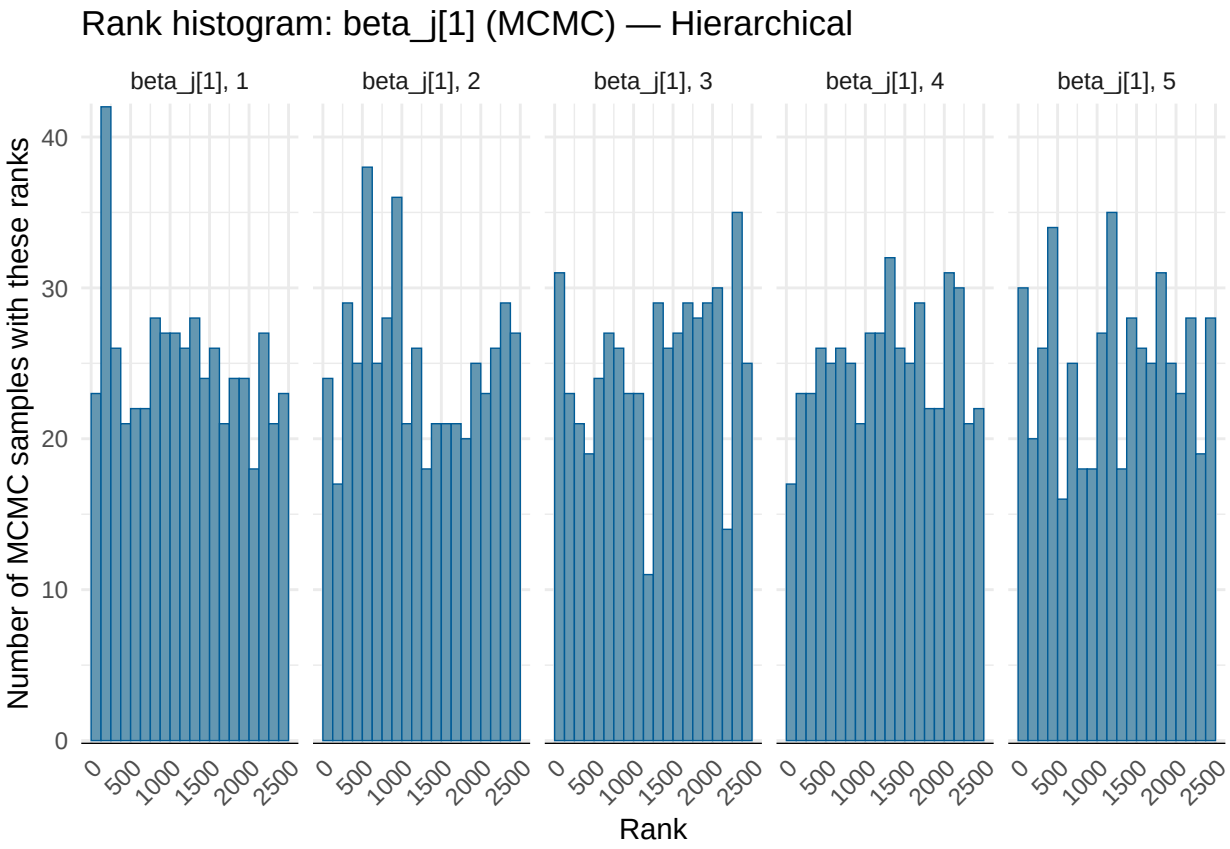

```
# Rank histograms for MCMC to assess uniform mixing across chains
dir.create("plots/rank_histograms_Hierarchical", recursive = TRUE, showWarnings = FALSE)

r1 <- mcmc_rank_hist(fit_hierarchical$draws("alpha")) +
  ylab("Number of MCMC samples with these ranks") +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1)) +
  labs(title = "Rank histogram: alpha (MCMC) - Hierarchical")
ggsave("plots/rank_histograms_Hierarchical/rank_mcmc_Hierarchical_alpha.png", r1, width = 8, height = 4

r2 <- mcmc_rank_hist(fit_hierarchical$draws("mu_beta")) +
  ylab("Number of MCMC samples with these ranks") +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1)) +
  labs(title = "Rank histogram: mu_beta (MCMC) - Hierarchical")
ggsave("plots/rank_histograms_Hierarchical/rank_mcmc_Hierarchical_mu_beta.png", r2, width = 8, height =

r3 <- mcmc_rank_hist(fit_hierarchical$draws("sigma_beta")) +
  ylab("Number of MCMC samples with these ranks") +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1)) +
  labs(title = "Rank histogram: sigma_beta (MCMC) - Hierarchical")
ggsave("plots/rank_histograms_Hierarchical/rank_mcmc_Hierarchical_sigma_beta.png", r3, width = 8, heigh

r4 <- mcmc_rank_hist(fit_hierarchical$draws("beta_j[1]")) +
  ylab("Number of MCMC samples with these ranks") +
```

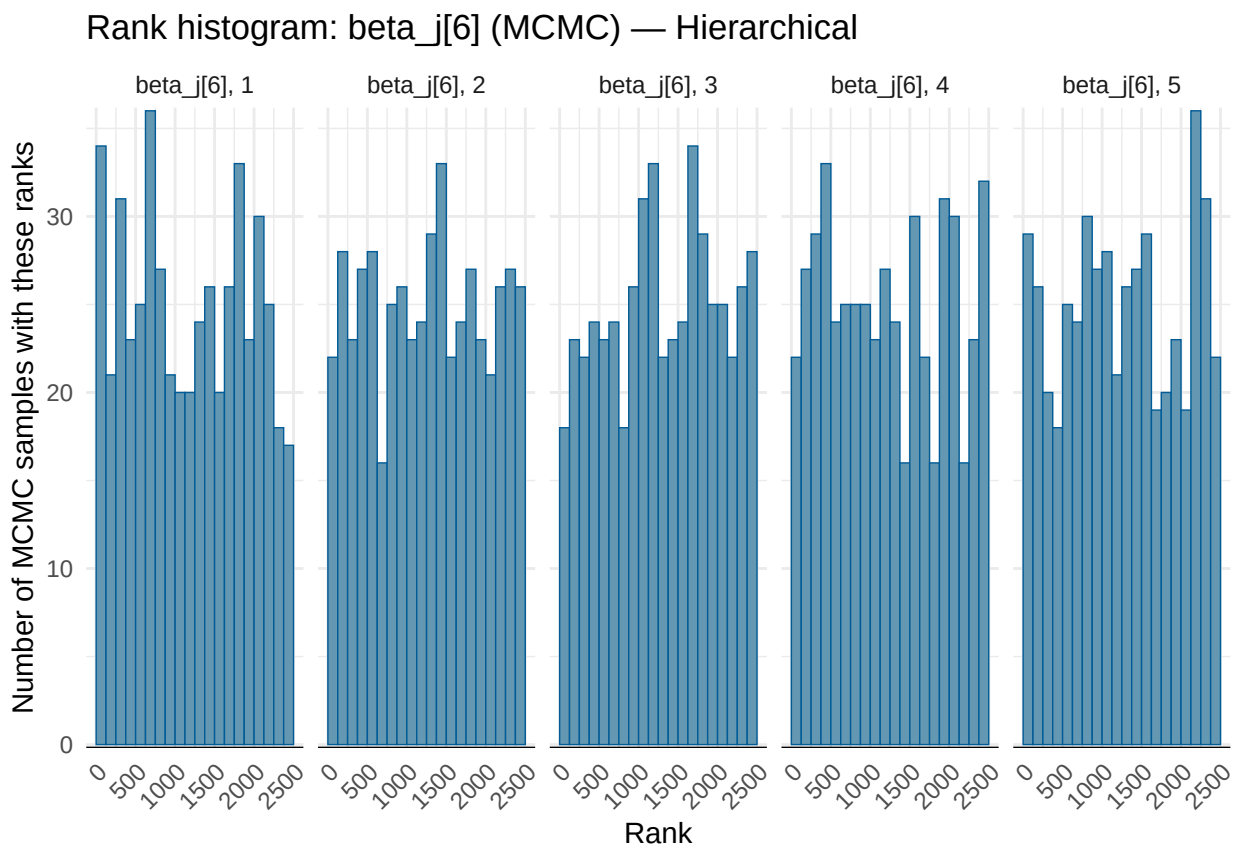
```
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1)) +
  labs(title = "Rank histogram: beta_j[1] (MCMC) - Hierarchical")
ggsave("plots/rank_histograms_Hierarchical/rank_mcmc_Hierarchical_beta_j1.png", r4, width = 8, height =

r5 <- mcmc_rank_hist(fit_hierarchical$draws("beta_j[6]")) +
  ylab("Number of MCMC samples with these ranks") +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1)) +
  labs(title = "Rank histogram: beta_j[6] (MCMC) - Hierarchical")
ggsave("plots/rank_histograms_Hierarchical/rank_mcmc_Hierarchical_beta_j6.png", r5, width = 8, height =

r1; r2; r3; r4; r5
```



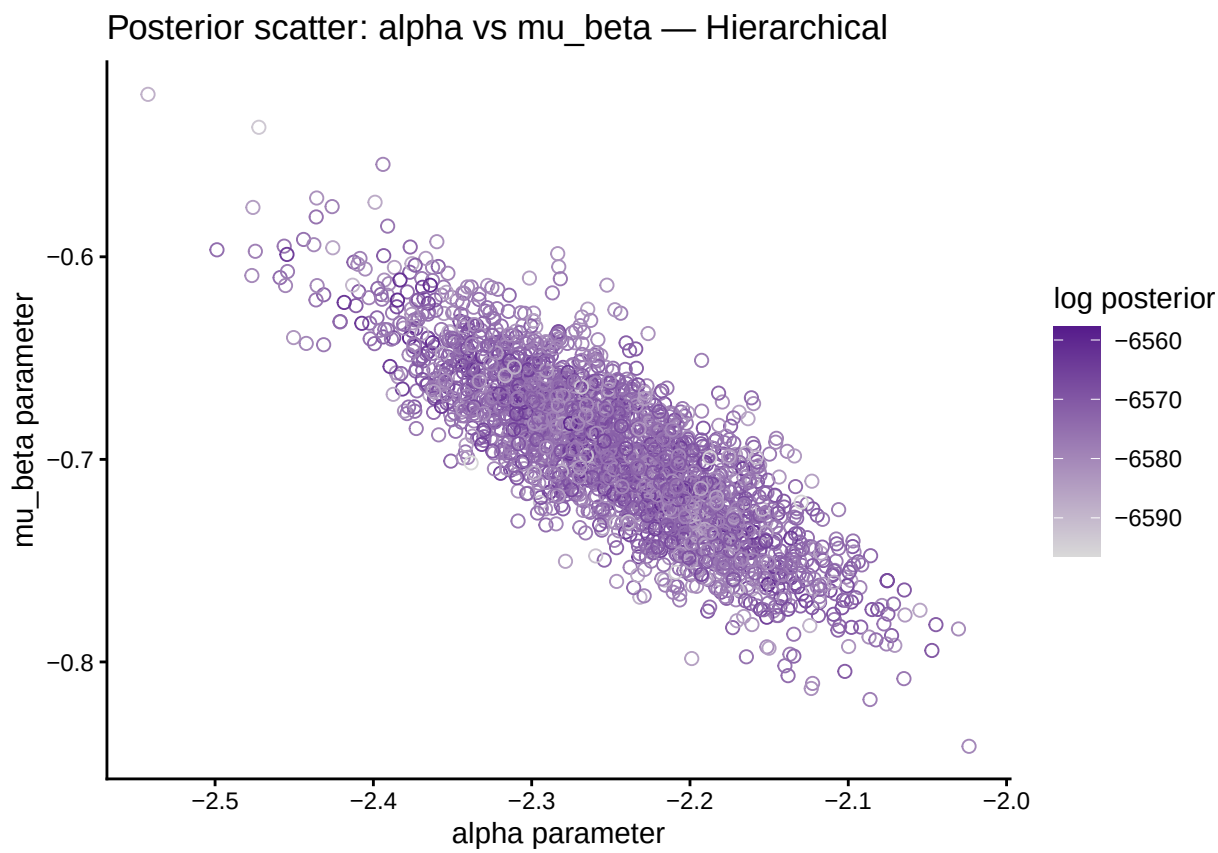

Rank histogram: mu_beta (MCMC) — Hierarchical
mu_beta, 1
mu_beta, 2
mu_beta, 3
mu_beta, 4
mu_beta, 5
Number of MCMC samples with these ranks
0
10
20
30
0
500
1000
1500
2000
2500
Rank


Rank histogram: sigma_beta (MCMC) — Hierarchical
sigma_beta, 1
sigma_beta, 2
sigma_beta, 3
sigma_beta, 4
sigma_beta, 5
Number of MCMC samples with these ranks
0
10
20
30
0
500
1000
1500
2000
2500
Rank

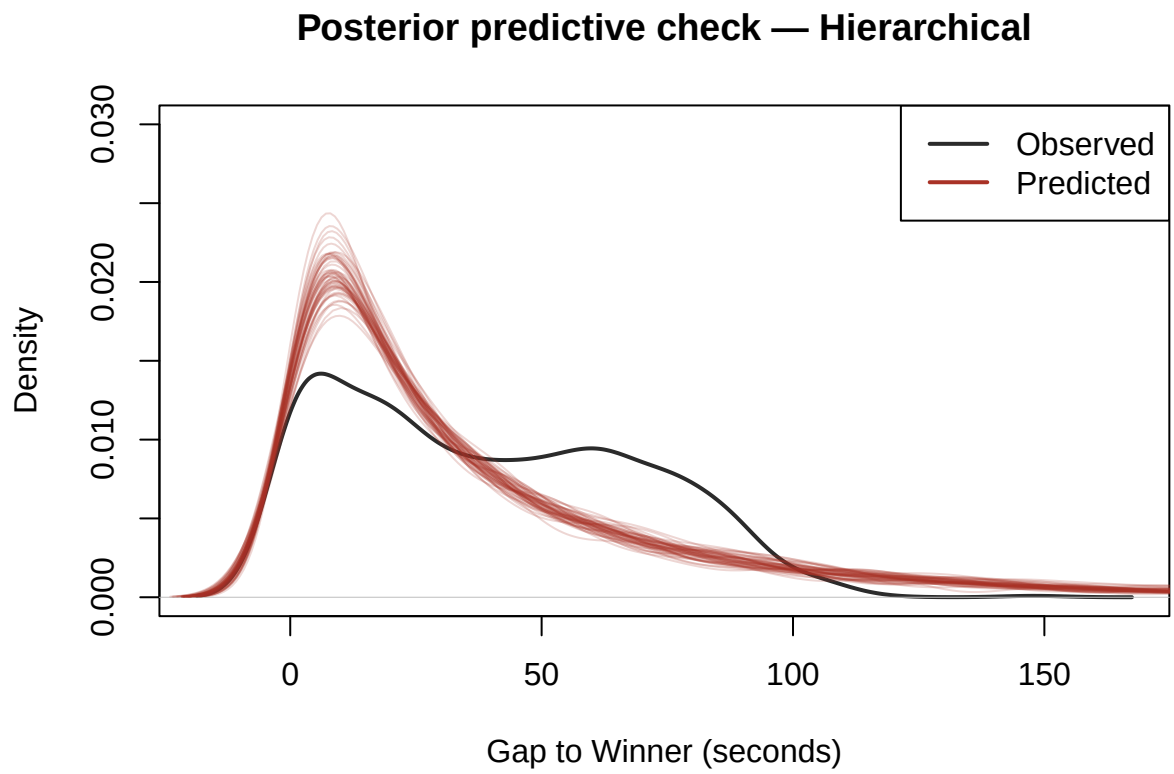

Rank histogram: beta_j[1] (MCMC) — Hierarchical
beta_j[1], 1
beta_j[1], 2
beta_j[1], 3
beta_j[1], 4
beta_j[1], 5
Number of MCMC samples with these ranks
0
10
20
30
40
0
500
1000
1500
2000
2500
Rank



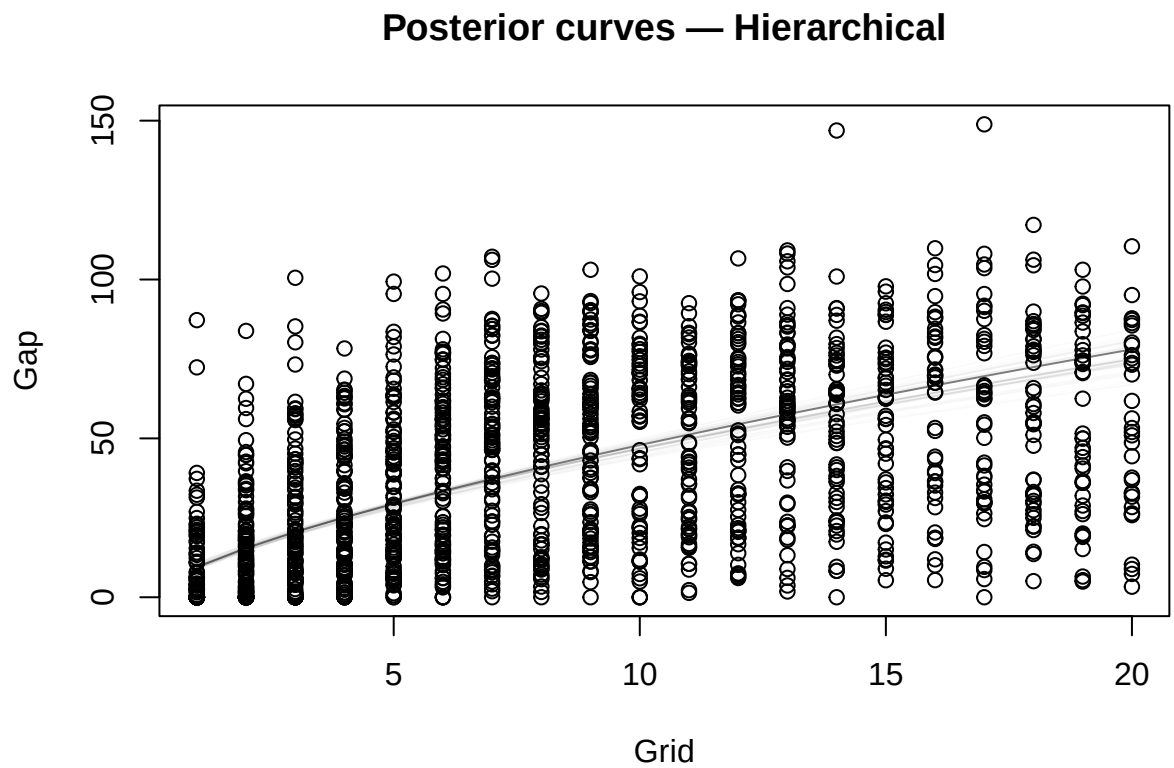

```r
# Posterior scatter plot of alpha vs mu_beta colored by log posterior
dir.create("plots/posterior_Hierarchical", recursive = TRUE, showWarnings = FALSE)

hier_draws_df <- fit_hierarchical$draws(format = "df")

s1 <- ggplot(hier_draws_df,
       aes(x = alpha, y = mu_beta, color = lp__)) +
  geom_point(shape = 1, size = 2) +
  scale_color_gradient(low = "grey85", high = "purple4") +
  labs(
    title = "Posterior scatter: alpha vs mu_beta - Hierarchical",
    x     = "alpha parameter",
    y     = "mu_beta parameter",
    color = "log posterior"
  ) +
  theme_classic()
ggsave("plots/posterior_Hierarchical/posterior_scatter_Hierarchical.png", s1, width = 8, height = 6)

s1
```




```r
# Posterior predictive check - observed vs predicted gap density
dir.create("plots/ppc_Hierarchical", recursive = TRUE, showWarnings = FALSE)

plot(density(data$gap),
     main = "Posterior predictive check - Hierarchical",
     xlab = "Gap to Winner (seconds)",
     col  = "#2b2b2b", lwd = 2,
     ylim = c(0, 0.03))

for (i in sample(1:nrow(hier_draws_df), 50)) {
  rate_pred <- exp(hier_draws_df$alpha[i] +
                   hier_draws_df$mu_beta[i] * log(data$grid))
  gap_pred  <- rexp(nrow(data), rate = rate_pred)
  lines(density(gap_pred), col = adjustcolor("#a93226", alpha.f = 0.2))
}

legend("topright",
       legend = c("Observed", "Predicted"),
       col    = c("#2b2b2b", "#a93226"),
       lwd    = 2)
```




```r
# Posterior mean curves overlaid on observed data weighted by log posterior
lp           <- hier_draws_df$lp__
norm_weights <- exp(lp - max(lp))
norm_weights <- norm_weights / sum(norm_weights)
norm_weights <- norm_weights / max(norm_weights)  # scale to [0,1]

xs <- seq(1, 20)

plot(data$grid, data$gap, xlab = "Grid", ylab = "Gap",
     main = "Posterior curves - Hierarchical")

for (i in 1:nrow(hier_draws_df)) {
  alpha    <- hier_draws_df$alpha[i]
  mu_beta  <- hier_draws_df$mu_beta[i]
  mean_gap <- exp(-(alpha + mu_beta * log(xs)))
  lines(xs, mean_gap, col = rgb(0, 0, 0, alpha = norm_weights[i] * 0.5))
}
```

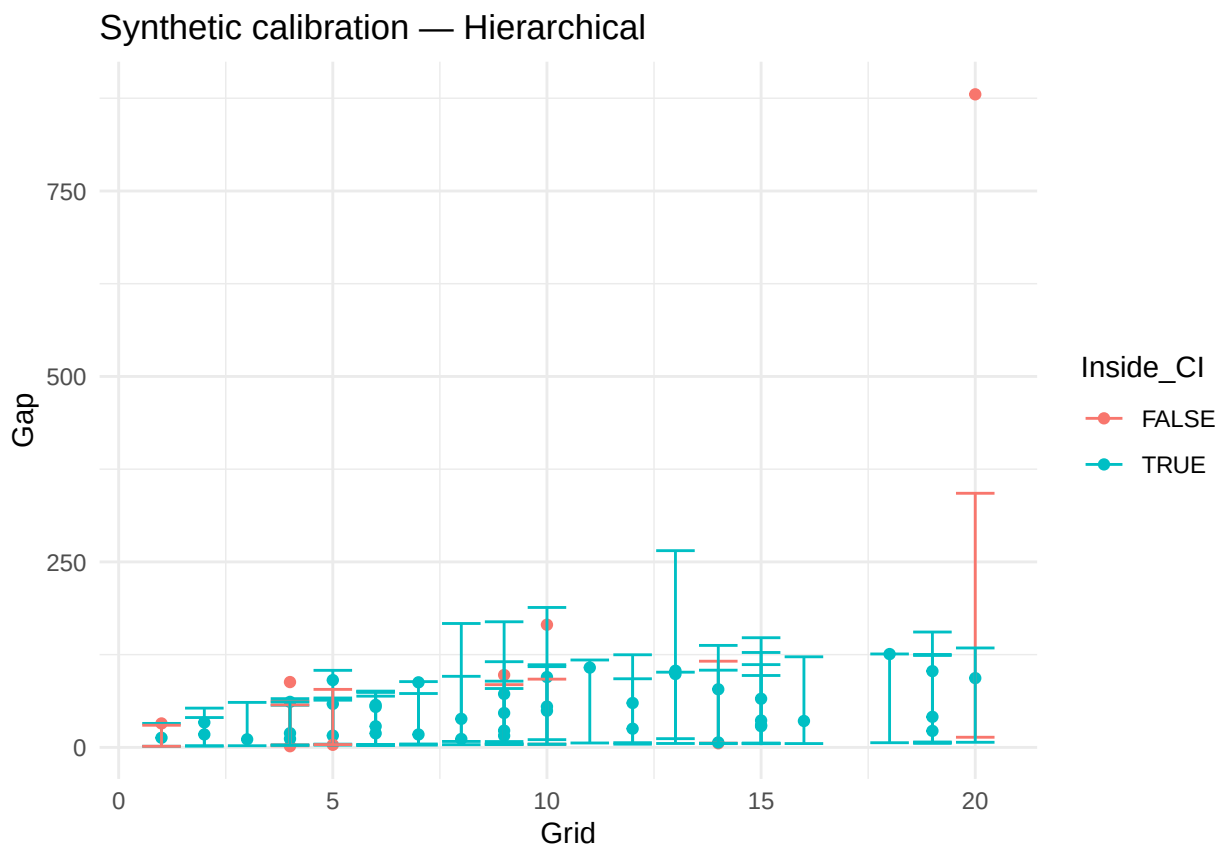



```r
# Simulate synthetic hierarchical F1 gap data from prior predictive distribution
alpha_mean = -2.5
alpha_sd   = 1
beta_mean  = -0.5
beta_sd    = 0.5

simulate_data_hier = function(n_circuits = 5, n_per_grid = 10) {
  alpha_sim      <- rnorm(1, alpha_mean, alpha_sd)
  mu_beta_sim    <- rnorm(1, beta_mean, beta_sd)
  sigma_beta_sim <- rexp(1, 1)
  beta_j_sim     <- rnorm(n_circuits, mu_beta_sim, sigma_beta_sim)

  grid_rep    <- rep(rep(1:20, each = n_per_grid), n_circuits)
  circuit_rep <- rep(1:n_circuits, each = 20 * n_per_grid)

  rate    <- exp(alpha_sim + beta_j_sim[circuit_rep] * log(grid_rep))
  gap_sim <- rexp(length(rate), rate = rate)

  return(data.frame(
    alpha      = alpha_sim,
    mu_beta    = mu_beta_sim,
    sigma_beta = sigma_beta_sim,
    grid       = grid_rep,
    gap        = gap_sim,
    circuit_id = circuit_rep
  ))
```

```r
}
```

```r
# Calibration loop on synthetic data to verify model recovers true parameters
simulated_data       <- simulate_data_hier()
validation_obs_debug <- sample(1:nrow(simulated_data), 50, replace = FALSE)
ci_limits_debug      <- matrix(0, nrow = 50, 2)

for (i in 1:length(validation_obs_debug)) {
  i_test <- validation_obs_debug[i]
  train_test_debug_dta = list(
    N            = nrow(simulated_data) - 1,
    J            = 5,
    circuit_id   = simulated_data$circuit_id[-i_test],
    grid         = simulated_data$grid[-i_test],
    gap          = simulated_data$gap[-i_test],
    grid_pred    = simulated_data$grid[i_test],
    circuit_pred = simulated_data$circuit_id[i_test]
  )
  mcmc_results_debug = mod_hierarchical$sample(
    seed             = i,
    data             = train_test_debug_dta,
    output_dir       = "stan_out_hierarchical",
    refresh          = 0,
    show_messages    = FALSE,
    show_exceptions  = FALSE,
    iter_warmup      = 500,
    iter_sampling    = 500,
    chains           = 1
  )
  samples_debug        <- as.vector(mcmc_results_debug$draws("gap_pred"))
  ci_limits_debug[i, ] <- quantile(samples_debug, ci_plims)
}
```

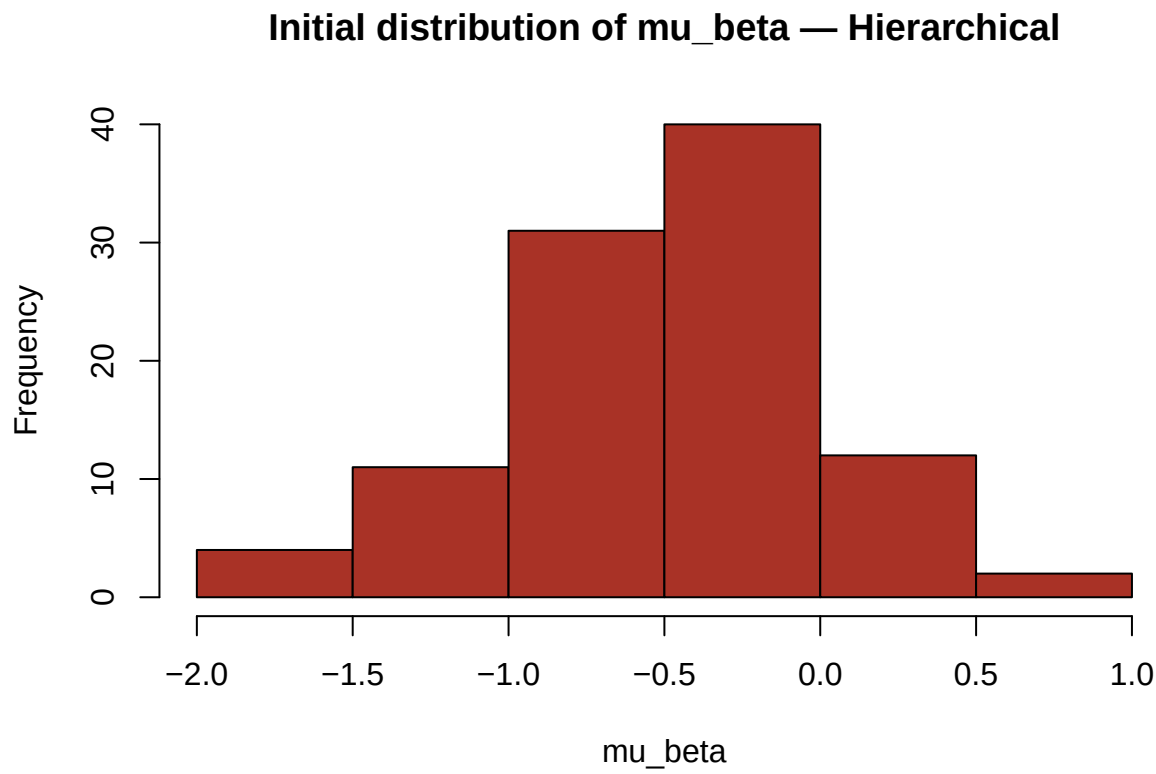
```r
# Check calibration coverage on synthetic data
merged_simulated_data = simulated_data[validation_obs_debug, ] |>
  bind_cols(data.frame(CI_L = ci_limits_debug[, 1], CI_R = ci_limits_debug[, 2])) |>
  mutate(Inside_CI = (gap >= CI_L & gap <= CI_R))

merged_simulated_data |>
  ggplot(aes(x = grid, y = gap, ymin = CI_L, ymax = CI_R, color = Inside_CI)) +
  geom_point() +
  geom_errorbar() +
  theme_minimal() +
  labs(title = "Synthetic calibration - Hierarchical",
       x = "Grid", y = "Gap")
```




```r
mean(merged_simulated_data$Inside_CI)
```

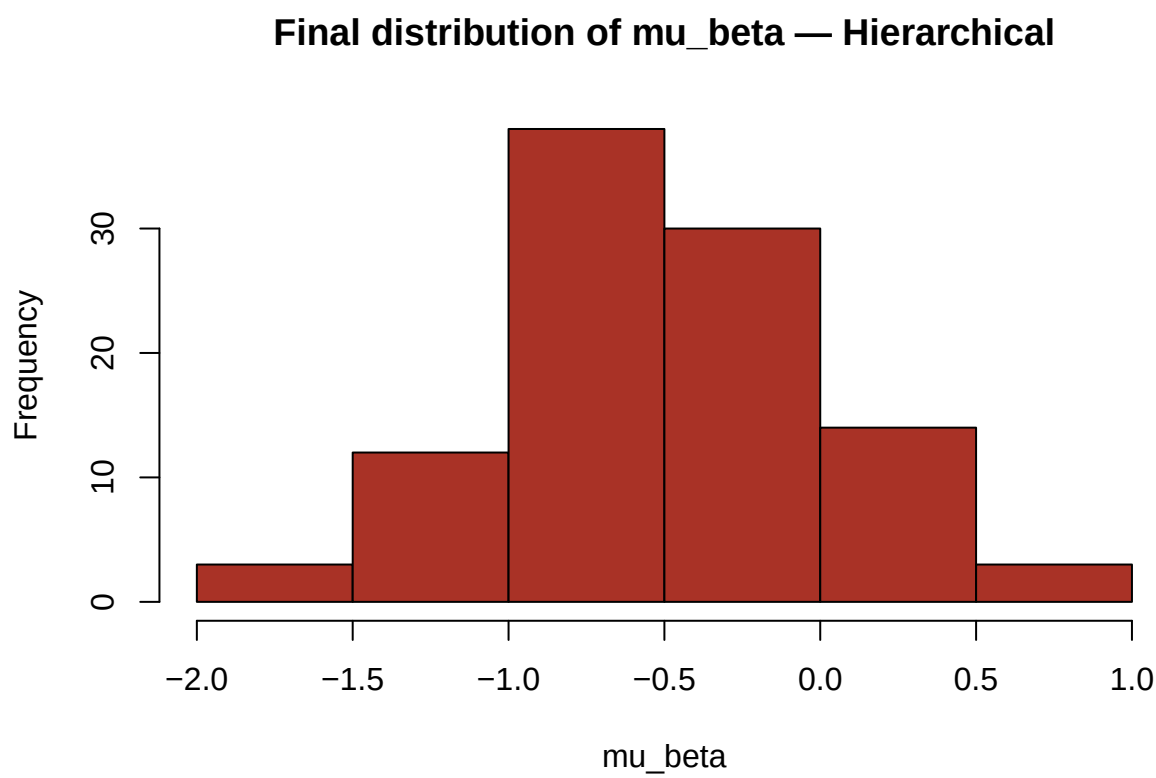
```
## [1] 0.84
```

```r
# Initial distribution of mu_beta sampled from prior
initial_dist_df = data.frame()
for (i in 1:100) {
  forward_values = simulate_data_hier()
  initial_dist_df = rbind(initial_dist_df,
                          data.frame(alpha   = forward_values$alpha[1],
                                     mu_beta = forward_values$mu_beta[1]))
}
hist(initial_dist_df$mu_beta, col = "#a93226",
     main = "Initial distribution of mu_beta - Hierarchical",
     xlab = "mu_beta")
```




```r
# Final distribution of mu_beta after MCMC transition on hierarchical synthetic data
final_dist_df = data.frame()
for (i in 1:100) {
  forward_values = simulate_data_hier()
  dta = list(
    N            = nrow(forward_values),
    J            = 5,
    circuit_id   = forward_values$circuit_id,
    grid         = forward_values$grid,
    gap          = forward_values$gap,
    grid_pred    = 10,
    circuit_pred = 1
  )
  curr_fit = mod_hierarchical$sample(
    seed            = i,
    data            = dta,
    refresh         = 0,
    show_messages   = FALSE,
    show_exceptions = FALSE,
    output_dir      = "stan_out_hierarchical",
    chains          = 1,
    iter_warmup     = 50,
    iter_sampling   = 50,
    init            = list(list(
      alpha      = forward_values$alpha[1],
      mu_beta    = forward_values$mu_beta[1],
```

```r
      sigma_beta = forward_values$sigma_beta[1],
      beta_j     = array(rnorm(5, forward_values$mu_beta[1],
                                forward_values$sigma_beta[1]), dim = 5)
    ))
  )
  final_params <- curr_fit$draws(format = "df") |>
    tail(1) |>
    subset(select = c(alpha, mu_beta))
  final_dist_df = rbind(final_dist_df, final_params)
}
hist(final_dist_df$mu_beta, col = "#a93226",
     main = "Final distribution of mu_beta - Hierarchical",
     xlab = "mu_beta")
```

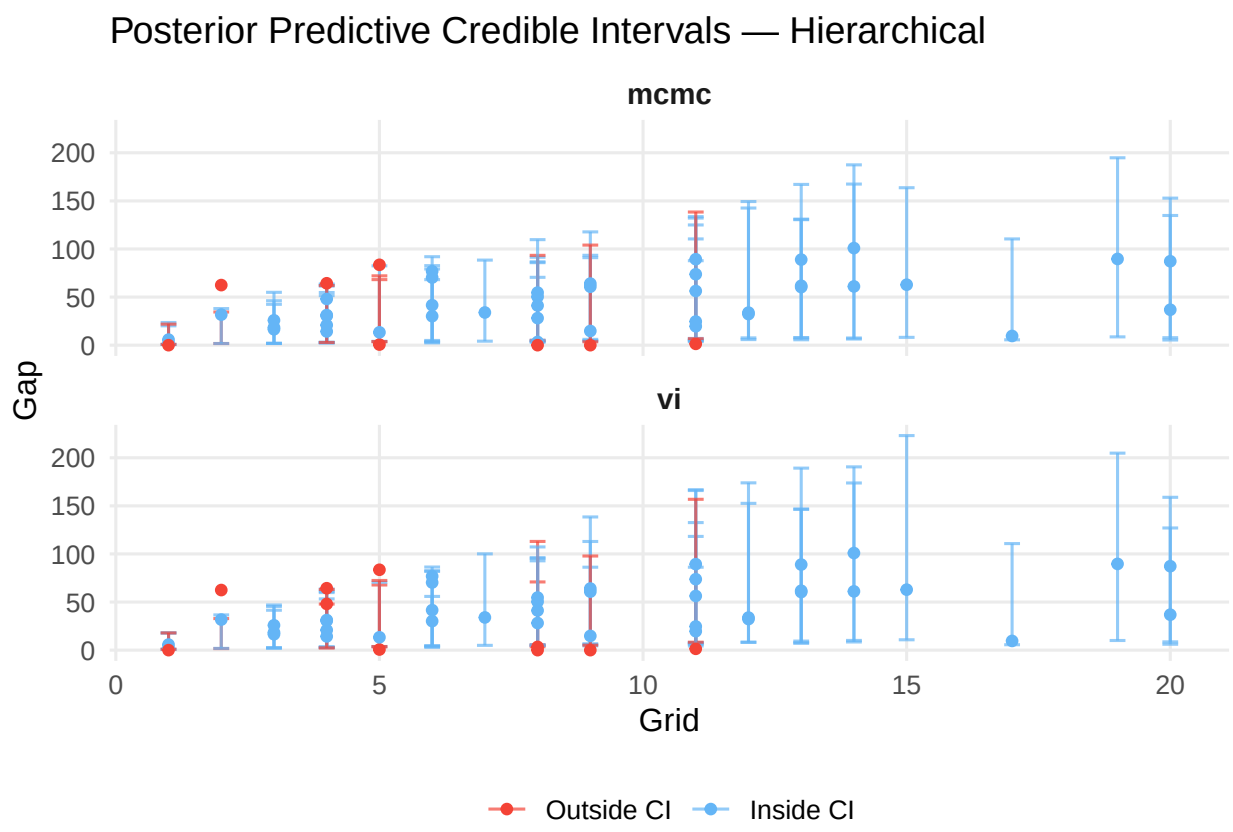


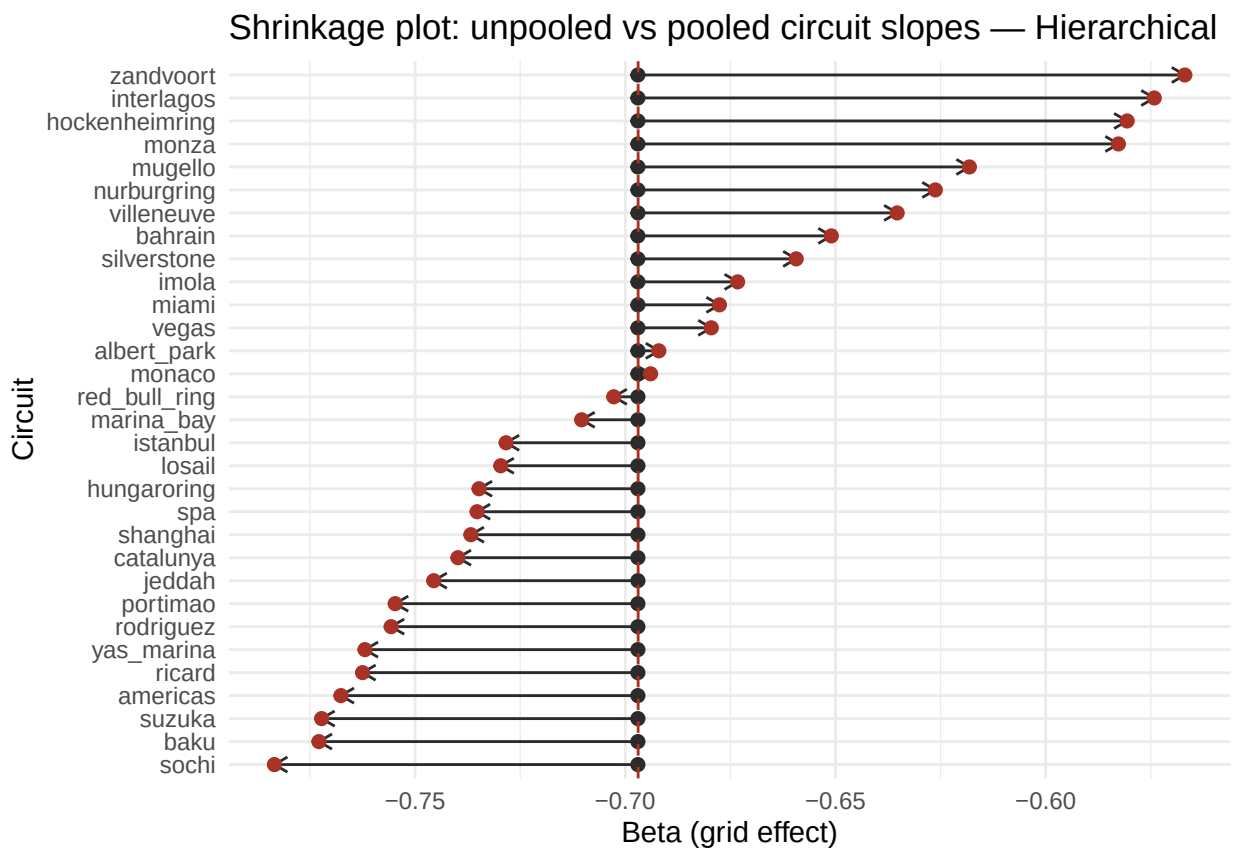

```r
# KS test comparing initial and final mu_beta distributions
ks.test(initial_dist_df$mu_beta, final_dist_df$mu_beta)
```

```
##
##  Asymptotic two-sample Kolmogorov-Smirnov test
##
## data:  initial_dist_df$mu_beta and final_dist_df$mu_beta
## D = 0.13, p-value = 0.3667
## alternative hypothesis: two-sided
```

```r
# Compare MCMC and VI credible intervals side by side
dir.create("plots/comparison_Hierarchical", recursive = TRUE, showWarnings = FALSE)
```

```r
merged_data_hier$method     = "mcmc"
merged_data_hier_var$method = "vi"

combined_merged_data_hier <- bind_rows(merged_data_hier, merged_data_hier_var)

c1 <- combined_merged_data_hier |>
  ggplot(aes(x = grid, y = gap, ymin = CI_L, ymax = CI_R, color = Inside_CI)) +
  geom_errorbar(width = 0.3, alpha = 0.7) +
  geom_point(size = 1.5) +
  scale_color_manual(
    values = c("TRUE" = "#64B5F6", "FALSE" = "#F44336"),
    labels = c("TRUE" = "Inside CI", "FALSE" = "Outside CI")
  ) +
  facet_wrap(~ method, ncol = 1) +
  theme_minimal(base_size = 12) +
  theme(
    legend.position  = "bottom",
    legend.title     = element_blank(),
    strip.text       = element_text(face = "bold", size = 11),
    panel.grid.minor = element_blank()
  ) +
  labs(title = "Posterior Predictive Credible Intervals - Hierarchical",
       x = "Grid", y = "Gap")

ggsave("plots/comparison_Hierarchical/mcmc_vi_comparison_Hierarchical.png",
       c1, width = 8, height = 8)

c1
```




```r
# Shrinkage plot showing partial pooling of circuit-specific slopes towards population mean
dir.create("plots/shrinkage_Hierarchical", recursive = TRUE, showWarnings = FALSE)

# get posterior means of beta_j for each circuit
beta_j_means <- fit_hierarchical$summary(
  paste0("beta_j[", 1:N_circuits, "]"), "mean"
)

# get raw unpooled beta per circuit (simple linear regression per circuit)
unpooled_betas <- data |>
  group_by(circuit, circuit_id) |>
  summarise(
    unpooled_beta = coef(lm(gap ~ log(grid)))[2],
    .groups = "drop"
  ) |>
  arrange(circuit_id)

shrinkage_df <- data.frame(
  circuit    = circuit_levels,
  circuit_id = 1:N_circuits,
  pooled     = beta_j_means$mean,
  unpooled   = rep(-0.697, N_circuits)
)

s2 <- ggplot(shrinkage_df) +
  geom_segment(aes(x = unpooled, xend = pooled,
```

```
                    y = reorder(circuit, pooled),
                    yend = reorder(circuit, pooled)),
                arrow = arrow(length = unit(0.2, "cm")),
                color = "#2b2b2b") +
  geom_point(aes(x = unpooled, y = reorder(circuit, pooled)),
             color = "#2b2b2b", size = 2) +
  geom_point(aes(x = pooled, y = reorder(circuit, pooled)),
             color = "#a93226", size = 2) +
  geom_vline(xintercept = mean(beta_j_means$mean),
             linetype = "dashed", color = "#a93226") +
  theme_minimal() +
  labs(title = "Shrinkage plot: unpooled vs pooled circuit slopes - Hierarchical",
       x     = "Beta (grid effect)",
       y     = "Circuit")

ggsave("plots/shrinkage_Hierarchical/shrinkage_plot_Hierarchical.png",
       s2, width = 8, height = 10)

s2
```